

Vinheria Agnello — Arquitetura de APIs e Microsserviços

Fase 6 — Atividade em grupo: documento consolidado

Integrantes e itens de responsabilidade

Nome completo	RM	Item de responsabilidade
Roger Viana Gonçalves de Alencar	97540	Item 1 — Identificação dos microsserviços e fronteiras de domínio
Kevin Benevides da Silva Romariz	557898	Item 2 — Diagrama de arquitetura
André Luiz dos Santos Flores	554952	Item 3 — Padrões de arquitetura e justificativas
Arthur Cei Corrêa	97781	Item 4 — Integração entre microsserviços (síncrono x assíncrono)
Yasmin Kimura	557413	Item 5 — Segurança, escalabilidade, resiliência e governança; consolidação do documento

Repositório oficial: <https://github.com/CodeSquad-FIAP/VinheriaAgnello>

Outubro de 2026

Sobre este documento

Este documento consolida, na ordem exigida pelo enunciado, as cinco seções da Fase 6 da Tarefa Anual: (1) identificação dos microsserviços e fronteiras de domínio; (2) diagrama de arquitetura; (3) padrões de arquitetura e justificativas; (4) integração entre serviços — comunicação síncrona e assíncrona; e (5) segurança, escalabilidade, resiliência e governança.

Cada seção foi escrita pelo integrante responsável, indicado no início da seção, a partir do arquivo Markdown correspondente no repositório do grupo ([Documentacao/Fase6/](#)). A consolidação manteve o conteúdo das partes e padronizou a numeração de seções, tabelas e figuras e as remissões internas, que agora apontam para as seções deste documento em vez de para os arquivos de origem.

As Figuras 1 a 4 vêm do arquivo [P2_arquitetura.drawio](#), que tem quatro páginas (visão geral e três detalhes). Os registros de revisão de cada seção e as instruções para regenerar as figuras estão nos apêndices.

Sumário

1. Identificação dos Microsserviços e Fronteiras de Domínio

- 1.1 Serviços candidatos a microsserviço
 - 1.1.1 Regras de posse dos dados (uma entidade, um dono)
 - 1.1.2 Sobre a contagem
 - 1.1.3 Critérios de aceite — conferência
 - 1.1.4 Dono do dado por entidade
- 1.2 Fronteiras de domínio
- 1.3 O que NÃO vira microsserviço (e por quê)
- 1.4 Do repositório ao serviço
- 1.5 Reconciliação com os domínios do enunciado
- 1.6 Glossário de nomes
- 1.7 Pontos a confirmar com o grupo

2. Diagrama de Arquitetura

- 2.1 Como ler o diagrama
- 2.2 Legenda
- 2.3 Conferência dos critérios de aceite
- 2.4 Figuras de detalhe e tamanho de impressão
- 2.5 Pontos a confirmar com o grupo

3. Padrões de Arquitetura e Justificativas

- 3.1 Padrões obrigatórios
 - 3.1.1 API Gateway Pattern
 - 3.1.2 Service Discovery Pattern
 - 3.1.3 Database per Service Pattern
 - 3.1.4 Saga Pattern
 - 3.1.4.1 Coreografia x orquestração
 - 3.1.4.2 Escolha para a Vinheria
- 3.2 Padrões adicionais
 - 3.2.1 Strangler Fig Pattern
 - 3.2.2 Cache (Redis) + API Composition
 - 3.2.3 Service Mesh com mTLS
- 3.3 Padrões avaliados e não incluídos nesta versão
- 3.4 Conferência dos critérios de aceite
- 3.5 Dependências e fronteiras com as outras partes
- 3.6 Pontos a confirmar com o grupo

4. Integração entre Serviços: Comunicação Síncrona x Assíncrona

- 4.1 Regra de decisão
- 4.2 Matriz de comunicação
- 4.3 Catálogo de eventos e tópicos
- 4.4 Fluxos ponta a ponta
 - 4.4.1 Compra de vinho
 - 4.4.2 Alerta de temperatura da adega
 - 4.4.3 Rastreabilidade e consulta do dia a dia
- 4.5 Conferência dos critérios de aceite
- 4.6 Consistência com as seções 1, 2 e 5

5. Segurança, Escalabilidade, Resiliência e Governança

- 5.1 Escopo e ligação com as demais partes
- 5.2 Retrato do sistema atual (evidências do repositório)

5.3 Autenticação

5.3.1 Padrão escolhido: OAuth 2.0 / OpenID Connect com Keycloak

5.3.2 Onde o token é validado: no API Gateway

5.3.3 Fluxos por tipo de cliente

5.3.4 Comunicação entre serviços

5.3.5 Boas práticas de API aplicadas

5.4 Autorização (RBAC + ABAC)

5.5 Escalabilidade

5.6 Resiliência

5.6.1 Padrões de falha aplicados

5.6.2 Fluxo de compensação (ligação com o Saga da seção 3)

5.6.3 Continuidade e deploy

5.7 Observabilidade

5.8 Segurança da informação, segredos e LGPD

5.8.1 Gestão de segredos

5.8.2 LGPD

5.8.3 Privacidade desde a concepção

5.9 Governança

5.9.1 Contrato como fonte da verdade

5.9.2 Versionamento e depreciação

5.9.3 Catálogo de serviços e papéis de nome

5.9.4 ADRs — Architecture Decision Records

5.10 Riscos residuais e pendências

5.11 Fechamento

Apêndice A — Divisão de tarefas

Apêndice B — Registro de revisão consolidado

Apêndice C — Como regenerar as figuras

1. Identificação dos Microserviços e Fronteiras de Domínio

Responsável: Roger Viana Gonçalves de Alencar — RM 97540 · Período: 16/09/2026 a 21/09/2026 (congelamento) · Versão 1.1, com revisão de consolidação de 22/09/2026

1.1 Serviços candidatos a microsserviço

O ponto de partida foi o que já existe no repositório do grupo: a API .NET de estoque, com as entidades **Vinho**, **Lote**, **Fornecedor**, **Categoria** e **TransacaoEstoque**; o monolito em JSP/Servlet; o app Android com Room; a cadeia Arduino, MQTT e Node-RED; e o par CSV com dashboard no Tableau. A partir disso chegamos a 12 serviços: **9 de núcleo**, que cobrem a cadeia de valor da vinícola do início ao fim, e **3 satélites** — identidade, notificações e analytics — que dão suporte a todos os domínios sem pertencer a nenhum deles em particular.

Tabela 1 — Serviços candidatos a microsserviço: responsabilidade única, dados e consumidores

Serviço	Classificação	Responsabilidade única	Dados dos quais é dono	Quem consome
ms-catalogo	Núcleo	Expor e manter o catálogo de vinhos, categorias e fichas de harmonização para os canais de venda.	Vinho (ficha comercial, teor, mídia) · Categoria · Regra de Harmonização	ms-pedidos · Web / App (canais) · ms-analytics
ms-producao	Núcleo	Registrar o ciclo produtivo do vinho, da colheita à fermentação e armazenagem, por safra.	Safra · Colheita · Fermentação · Armazenagem	ms-lotes · ms-qualidade · ms-analytics
ms-lotes	Núcleo	Garantir a rastreabilidade individual de cada lote, do engarrafamento ao consumidor.	Lote · Genealogia do Lote · Código de Rastreo / QR · Histórico do lote	ms-estoque · ms-catalogo · ms-qualidade · App do consumidor
ms-estoque	Núcleo	Manter a posição de estoque em tempo real e registrar movimentações e reservas.	Posição de Estoque · MovimentacaoEstoque · Reserva	ms-pedidos · ms-fornecedores · App mobile · ms-analytics
ms-pedidos	Núcleo	Orquestrar o ciclo de vida do pedido, do carrinho à confirmação de entrega.	Carrinho · Pedido · Item do Pedido · Status de Separação / Entrega	ms-pagamentos · ms-estoque · ms-notificacoes · ms-clientes
ms-fornecedores	Núcleo	Gerir o cadastro de fornecedores e o fluxo de compras e recebimento.	Fornecedor · PedidoCompra · Recebimento	ms-estoque · ms-producao · ms-analytics
ms-qualidade	Núcleo	Monitorar condições ambientais da adega e registrar análises e alertas.	Leitura de Temperatura / Umidade · Análise Laboratorial · Alerta	ms-estoque · ms-producao · ms-analytics · Painel operacional
ms-clientes	Núcleo	Manter cadastro, perfil e consentimento LGPD de cada cliente.	Cliente · Perfil · Consentimento LGPD	ms-pedidos · ms-identidade · ms-notificacoes · ms-analytics
ms-pagamentos	Núcleo	Processar cobrança, conciliação financeira e estornos das vendas.	Cobrança · Transação · Conciliação · Estorno	ms-pedidos · ms-analytics · ms-notificacoes
ms-identidade	Satélite	Autenticar e autorizar usuários e serviços via OAuth2/OIDC e RBAC.	Usuário / Credencial · Papel / Permissão (RBAC) · Token / Sessão	ms-pedidos · ms-clientes · ms-fornecedores (painel) · App mobile

ms-notificacoes	Satélite	Disparar e rastrear comunicações por e-mail, WhatsApp e push.	Template de Mensagem · Envio (status) · Preferência de canal	ms-pedidos · ms-qualidade · ms-pagamentos · ms-clientes
ms-analytics	Satélite	Consolidar indicadores de venda e produção para previsão e dashboards.	Indicador Consolidado · Modelo de Previsão	Dashboard (BI) · Gestão / Direção

1.1.1 Regras de posse dos dados (uma entidade, um dono)

Para que a seção 3 (Database per Service) e o diagrama da seção 2 não encontrem posse duplicada, valem estas quatro regras. Nenhum serviço compartilha tabela com outro: cada entidade tem um único dono.

1. **Safra pertence ao ms-producao.** O ms-catalogo guarda a ficha comercial do vinho (teor, harmonização, mídia) e apenas referencia a safra de exibição pelo `safraId`; não é dono do registro de safra. O ms-lotes também referencia `safraId` ao classificar o lote.
2. **Nome canônico da movimentação.** O que hoje existe como `TransacaoEstoque` no .NET passa a se chamar `MovimentacaoEstoque` no alvo — é a mesma entidade, renomeada na migração. Ela pertence ao ms-estoque e não se duplica em outro serviço.
3. **Só o ms-estoque escreve no agregado de estoque.** Posição, movimentação e reserva são escritas exclusivamente por ele. A entrada gerada pelo recebimento de uma compra nasce no ms-fornecedores (dono do `Recebimento`) e chega ao ms-estoque como evento; **quem registra o movimento de entrada é o ms-estoque.** Nenhum outro serviço escreve posição de estoque.
4. **Cópias derivadas são declaradas como tal.** O histórico de compras do ms-clientes e o dataset histórico do ms-analytics são *read-models* (cópias somente-leitura) alimentados por evento a partir do ms-pedidos; não são posse de dado. O dono do pedido continua sendo o ms-pedidos.

1.1.2 Sobre a contagem

O enunciado pede **os principais serviços** do sistema e exemplifica quatro domínios (produção, rastreamento de lotes, pedidos/compras/fornecedores/estoque e qualidade). A decomposição do grupo chegou a **12 serviços: 9 de núcleo + 3 satélites**, pelos critérios de fronteira da seção 1.2:

- **Núcleo (9)** — os que carregam a cadeia de valor da vinícola: ms-catalogo, ms-producao, ms-lotes, ms-estoque, ms-pedidos, ms-fornecedores, ms-qualidade, ms-clientes e ms-pagamentos;
- **Satélites (3)** — capacidades de plataforma que servem a todos os domínios sem pertencer a nenhum: ms-identidade, ms-notificacoes e ms-analytics.

Optamos por não inflar a lista para arredondar um número: entre criar um serviço a mais só para fechar uma contagem e manter apenas serviços com responsabilidade genuinamente coesa, ficamos com a segunda opção — o mesmo critério usado para descartar os seis itens da seção 1.3. Se o grupo quiser um serviço de núcleo a mais, o único candidato natural é extrair o carrinho do ms-pedidos (avaliado e descartado na seção 1.3 por ser estado efêmero do mesmo agregado); a decisão está registrada na seção 1.7.

1.1.3 Critérios de aceite — conferência

Tabela 2 — Conferência dos critérios do item 1

Critério do enunciado	Onde é atendido
Cada serviço tem responsabilidade única e não compartilha tabela com outro	Coluna "Responsabilidade única" da tabela da seção 1.1 e regras de posse da seção 1.1.1

Cada serviço diz quem é o dono do dado (uma entidade por linha)	Coluna "Dados dos quais é dono" da tabela da seção 1.1, com as cópias derivadas declaradas na seção 1.1.1
Nomes padronizados (<code>ms-...</code>)	seção 1.6 — glossário congelado, 12 nomes, sem exceção
Seção curta do que ficou de fora e por quê	seção 1.3, com seis descartes justificados

1.1.4 Dono do dado por entidade

Cada entidade pertence a **um único** serviço, e esta é a lista que a seção 3 (*Database per Service*) usa como entrada. Onde existe leitura cruzada, ela acontece por API ou por evento — nunca por acesso direto ao banco do vizinho.

Tabela 3 — Dono do dado por entidade

Serviço	Entidades das quais é dono
<code>ms-catalogo</code>	Vinho (ficha comercial), Categoria, Regra de Harmonização, Mídia do Produto
<code>ms-producao</code>	Safra, Colheita, Fermentação, Armazenagem
<code>ms-lotes</code>	Lote, Genealogia do Lote, Código de Rastreo / QR, Histórico do Lote
<code>ms-estoque</code>	Posição de Estoque, MovimentacaoEstoque, Reserva
<code>ms-pedidos</code>	Carrinho, Pedido, Item do Pedido, Status de Separação / Entrega
<code>ms-fornecedores</code>	Fornecedor, PedidoCompra, Recebimento
<code>ms-qualidade</code>	Leitura de Temperatura / Umidade, Análise Laboratorial, Alerta
<code>ms-clientes</code>	Cliente, Perfil, Consentimento LGPD
<code>ms-pagamentos</code>	Cobrança, Transação, Conciliação, Estorno
<code>ms-identidade</code>	Usuário / Credencial, Papel / Permissão (RBAC), Token / Sessão
<code>ms-notificacoes</code>	Template de Mensagem, Envio (status), Preferência de Canal
<code>ms-analytics</code>	Indicador Consolidado, Modelo de Previsão

Fora dessa lista, duas coisas circulam como **projeção derivada**, não como posse: o histórico de compras em `ms-clientes` (projeção do que pertence ao `ms-pedidos`) e o dataset histórico em `ms-analytics` (cópia alimentada por evento). `Safra` aparece uma única vez, no `ms-producao`, conforme a regra 1 da seção 1.1.1.

1.2 Fronteiras de domínio

Cada corte que propomos aqui delimita o que a literatura de DDD chama de *Bounded Context*: um pedaço do sistema com vocabulário e regras próprias, que muda no seu próprio ritmo. Por isso ele não deveria dividir tabela, deploy nem time com o vizinho.

O que existe hoje no repositório é o contraexemplo do recorte que queremos: em `Web/src/main/java/br/com/fiap/vinheriaagnello/`, o catálogo (`web/VitrineServlet.java`, `web/VinhoServlet.java`) e o cadastro de clientes (`web/ClienteCadastroServlet.java`) compartilham o mesmo `repository/InMemoryDatabase.java` e os mesmos modelos de `model/` — mexer na vitrine obriga a testar o cadastro. Cada fronteira abaixo existe para não repetir esse acoplamento em escala de serviço. Os critérios usados em cada corte estão descritos a seguir — os 12 serviços aparecem justificados.

- **Catálogo e lotes parecem parecidos, mas mudam por razões diferentes.** O catálogo (`ms-catalogo`) muda por motivo comercial, como uma safra nova em destaque ou uma ficha de harmonização reescrita. Lotes (`ms-lotes`) muda por exigência regulatória, ligada à rastreabilidade do vinho. Juntar os dois num módulo só faria o time comercial testar regra fiscal toda vez que trocasse uma foto de produto.

- **Produção x Qualidade.** Um decide o que produzir e quando (`ms-producao`). O outro mede se o que já foi produzido está dentro do padrão prometido (`ms-qualidade`). A diferença de frequência é grande: a telemetria dos sensores gera uma leitura a cada poucos segundos, um volume que não faz sentido impor ao modelo transacional, bem mais lento, da produção.
- **Estoque e pedidos guardam verdades diferentes.** O estoque (`ms-estoque`) sabe quanto existe de fato na adega. Pedidos (`ms-pedidos`) sabe quanto já foi prometido a algum cliente. Se fossem o mesmo serviço, um pico de acesso ao catálogo em data comemorativa passaria a competir, na mesma escala, com a escrita do estoque, que não pode errar.
- **Pagamentos isolado.** Cobrança, conciliação e estorno lidam com dado sensível e exigem trilha de auditoria própria, no padrão PCI-DSS. Não há razão para replicar essa exigência em outro contexto. Isolar tudo em `ms-pagamentos` limita a um único serviço o raio de exposição desse dado.
- **Compras e recebimento x estoque.** `ms-fornecedores` é dono do documento de compra (`PedidoCompra`) e do `Recebimento` — a conferência física do que chegou, com preço e divergência. `ms-estoque` é dono da posição e do movimento. São verdades com donos diferentes: uma compra pode estar recebida e ainda não movimentada, e o inverso nunca pode acontecer. Por isso o movimento de entrada é escrito só pelo `ms-estoque`, a partir do evento de recebimento.
- **Fornecedores x pedidos.** `PedidoCompra` (suprimento: fornecedor, custo, prazo de entrega) e `Pedido` (venda: cliente, receita, entrega) têm ciclos, indicadores e interlocutores distintos. Chamar os dois de "pedido" no mesmo serviço é o caminho mais curto para relatório de compra entrando no faturamento — por isso o nome `PedidoCompra` fica no `ms-fornecedores` e o `Pedido` no `ms-pedidos`.
- **Clientes x Identidade.** Dado de cliente é negócio: cadastro, perfil e consentimento LGPD, que muda quando o cliente quer, com obrigação legal de exclusão. Credencial, papel e token são plataforma: mudam quando a política de segurança muda. Se fossem o mesmo serviço, uma alteração de política de senha passaria a mexer no cadastro comercial e o consentimento LGPD ficaria no caminho crítico do login.
- **Identidade e notificações não carregam nenhuma regra de negócio da vinícola:** são capacidades de plataforma, usadas por qualquer domínio. Por isso viraram satélites, e não núcleo — assim evitamos a tentação de grudar autenticação ou envio de e-mail na lógica de um domínio específico.
- **Analytics também é satélite, e lê, não escreve.** `ms-analytics` consolida indicadores a partir de eventos dos domínios e é dono apenas do indicador consolidado e do modelo de previsão. Se fosse núcleo, toda decisão de BI viraria dependência do caminho transacional — e um dashboard pesado poderia derrubar venda.

1.3 O que NÃO vira microsserviço (e por quê)

Um recorte generoso demais tem custo real: dilui responsabilidade e multiplica chamada de rede sem ganhar coesão em troca. Por isso descartamos de propósito os itens a seguir como serviços independentes.

- **Categoria isolada.** Tem poucos valores e quase nunca muda. Transformá-la num serviço à parte criaria uma chamada de rede para responder a algo que uma tabela de referência dentro do `ms-catalogo` resolve sozinha. É o caso do `Categoria.cs` atual.
- **A genealogia do lote não é um contexto de negócio à parte.** É só uma faceta do dado, um relacionamento entre lotes, e cabe perfeitamente dentro do `ms-lotes`.

- **Camada de apresentação.** O monolito `Web/src/main/java/br/com/fiap/vinheriaagnello/` (Servlet/JSP) e o app Android em `Mobile/app/src/main/java/com/example/myapplication/` continuam existindo, só que agora como clientes dos microsserviços (ou, no limite, um BFF, sigla de *Backend-for-Frontend*). Como não são donos de regra de negócio nem de dado, não entram nesta lista. A validação de sessão que hoje está em `AuthFilter.java` migra para o API Gateway.
- **O Node-RED segue fazendo o que já faz hoje:** traduzir e repassar telemetria do broker MQTT para o `ms-qualidade`, conforme `MQTT/node_red_vinheria_flow.json` e `MQTT/README_MQTT.md`. Não tem domínio de negócio nem dado próprio, então continua sendo infraestrutura de integração, não um serviço.
- **Um `ms-relatorios` à parte.** Duplicaria a posse de um dado que já pertence ao `ms-analytics`. Um relatório é só uma forma de exibir a mesma informação analítica, não um contexto de negócio novo.
- **Separar o carrinho do `ms-pedidos` também não se justifica agora.** O carrinho é um estado efêmero, anterior à transação, do mesmo agregado do pedido; isolá-lo cedo criaria consistência distribuída sem ganho real, um YAGNI clássico nesta fase do projeto.

1.4 Do repositório ao serviço

Nada disso veio de teoria abstrata. Cada corte foi ancorado em algo que o grupo já tinha construído, e a tabela abaixo mostra de onde veio cada decisão, com o caminho do arquivo no repositório.

Tabela 4 — Do repositório ao serviço: artefato existente e serviço herdeiro

Artefato no repositório	Serviço herdeiro / papel
<code>Web/src/VinheriaAgnello.Server/src/VinheriaAgnello.Domain/Entities/Vinho.cs</code> e <code>Categoria.cs</code>	<code>ms-catalogo</code>
<code>Web/src/VinheriaAgnello.Server/src/VinheriaAgnello.Domain/Entities/Lote.cs</code>	<code>ms-lotes</code>
<code>Web/src/VinheriaAgnello.Server/src/VinheriaAgnello.Domain/Entities/Fornecedor.cs</code>	<code>ms-fornecedores</code>
<code>Web/src/VinheriaAgnello.Server/src/VinheriaAgnello.Domain/Entities/TransacaoEstoque.cs</code>	<code>ms-estoque</code> (renomeado para <code>MovimentacaoEstoque</code>)
<code>Web/src/VinheriaAgnello.Server/src/VinheriaAgnello.API/Controllers/EstoqueController.cs</code> e <code>Program.cs</code>	<code>ms-estoque</code> (a API .NET de hoje é o serviço extraído)
<code>Web/src/main/java/br/com/fiap/vinheriaagnello/</code> (<code>web/VitrineServlet.java</code> , <code>web/VinhoServlet.java</code> , <code>web/ClienteCadastroServlet.java</code> , <code>model/repository/InMemoryDatabase.java</code>)	Cliente / BFF dos microsserviços; não é um serviço. Hoje concentra catálogo e clientes — consumidor futuro de <code>ms-catalogo</code> e <code>ms-clientes</code>
<code>Mobile/app/src/main/java/com/example/myapplication/data/local/</code> (<code>AppDatabase.kt</code> , <code>entity/Produto.kt</code> , <code>dao/ProdutoDao.kt</code>) e <code>ui/estoque/</code>	Cliente / BFF dos microsserviços; não é um serviço. Hoje é espelho local de estoque — consumidor futuro de <code>ms-estoque</code>
<code>Arduino/VinheriaSensores/VinheriaSensores.ino</code> + <code>MQTT/node_red_vinheria_flow.json</code> + <code>MQTT/README_MQTT.md</code>	<code>ms-qualidade</code> (o Node-RED atua como gateway de integração)

Web/historico_vendas-vinheria_agnello.csv + Web/vinheria_dashboard.twbx	ms-analytics
--	--------------

1.5 Reconciliação com os domínios do enunciado

O enunciado pede os principais serviços e exemplifica quatro domínios. A tabela abaixo é a correspondência entre cada exemplo do enunciado e os serviços propostos nesta entrega, com a razão de ter havido separação em mais de um serviço.

Tabela 5 — Correspondência entre os domínios do enunciado e os serviços propostos

Exemplo do enunciado	Serviços propostos	Por que não é um serviço só
Gestão de Produção — registro de colheitas, fermentação, armazenamento	ms-producao (a armazenagem é posição de estoque e fica no ms-estoque)	A safra e o ciclo produtivo são um domínio coeso; a posição física do vinho pertence ao estoque, que é escrito só por ele (regra 3 da seção 1.1.1)
Rastreamento de Vinhos — controle de lotes, histórico de produção	ms-lotes (a safra vem do ms-producao)	Lote, genealogia, código de rastreio e histórico são um domínio só, que muda por exigência regulatória — não pelo motivo comercial do catálogo (seção 1.2)
Gestão de Pedidos — controle de compras, fornecedores e estoque	ms-pedidos, ms-fornecedores e ms-estoque	São três verdades com donos diferentes: Pedido (venda, cliente, receita), PedidoCompra (suprimento, custo, prazo) e Posição de Estoque (o que existe de fato) — separação justificada nos cortes "Fornecedores × pedidos" e "Compras e recebimento × estoque" (seção 1.2)
Monitoramento de Qualidade — temperatura, umidade e análises laboratoriais	ms-qualidade	Um serviço só, com banco de série temporal: recebe a telemetria da adega e publica os alertas (seções 2 e 4)

O resultado da análise dos dois lados:

- **Sem correspondência direta no enunciado (6):** ms-catalogo (vitrine comercial dos vinhos), ms-clientes (cadastro, perfil e consentimento LGPD), ms-pagamentos (cobrança e conciliação), ms-identidade (credencial e RBAC), ms-notificacoes (e-mail, WhatsApp e push) e ms-analytics (indicadores e previsão) — todos ancorados em algo que já existe no repositório (seção 1.4).
- **Nomes:** nenhum serviço foi renomeado; o glossário da seção 1.6 é a mesma nomenclatura que o diagrama (seção 1.2), os padrões (seção 1.3) e a comunicação (seção 1.4) usam. Uma entidade muda de nome no alvo: TransacaoEstoque → MovimentacaoEstoque.
- **Cortados:** nenhum serviço. O carrinho foi avaliado e **não** promovido a serviço; permanece como agregado dentro do ms-pedidos (justificativa na seção 1.3).
- **Separação que o enunciado não pede explicitamente:** a **classificação núcleo/satélite** e a divisão entre ms-clientes (dado de cliente e LGPD) e ms-identidade (credencial e RBAC), que sem ela ficariam no mesmo serviço de plataforma.
- **Contagem:** 12 serviços (9 núcleo + 3 satélites), conforme a seção 1.1.2.

1.6 Glossário de nomes

Esta é a nomenclatura definitiva dos serviços. As seções 2, 3 e 4 deste documento, o diagrama de contexto e a comunicação entre serviços devem usar exatamente estes nomes.

Tabela 6 — Glossário de nomes dos serviços

Serviço	Classificação
---------	---------------

ms-catalogo	Núcleo
ms-producao	Núcleo
ms-lotes	Núcleo
ms-estoque	Núcleo
ms-pedidos	Núcleo
ms-fornecedores	Núcleo
ms-qualidade	Núcleo
ms-clientes	Núcleo
ms-pagamentos	Núcleo
ms-identidade	Satélite
ms-notificacoes	Satélite
ms-analytics	Satélite

Tópicos de evento seguem a lista já escrita na seção 5, para não haver duas grafias: pedido.criado, estoque.reservado, pagamento.aprovado, pagamento.recusado, lote.criado, qualidade.leitura, qualidade.alerta, notificacao.enviar e cliente.anonimizado. Formato <dominio>.<evento>, minúsculo, sem prefixo de turma; fila morta como <topico>.DLQ.

1.7 Pontos a confirmar com o grupo

Registrado em 22/09/2026, antes do início das seções 2 e 3:

- **Evento recebimento.confirmando** — proposto neste item para ligar o recebimento de compra (ms-fornecedores) ao ms-estoque. Ele **não** está na lista de tópicos já escrita na seção 5; a confirmação do nome fica com a seção 4 (comunicação entre serviços).
- **Serviços de núcleo** — 9 de núcleo e 3 satélites, total de 12 (seção 1.1.2). Se o grupo quiser um serviço de núcleo a mais, o candidato é extrair o carrinho do ms-pedidos.
- **MovimentacaoEstoque** — nome canônico da entidade que hoje existe como TransacaoEstoque.cs; a seção 3 usa este nome no Database per Service.

2. Diagrama de Arquitetura

Responsável: Kevin Benevides da Silva Romariz — RM 557898 · Período: 22/09/2026 a 28/09/2026 · Revisões de consolidação de 29/09/2026 e 01/10/2026 (Yasmin Kimura)

Vinheria Agnello — Arquitetura de Microsserviços

Fase 6 · Item 2 · 12 serviços da lista congelada do P1 (9 núcleo + 3 satélites) · Database per Service · API Gateway · Kafka · Strangler Fig

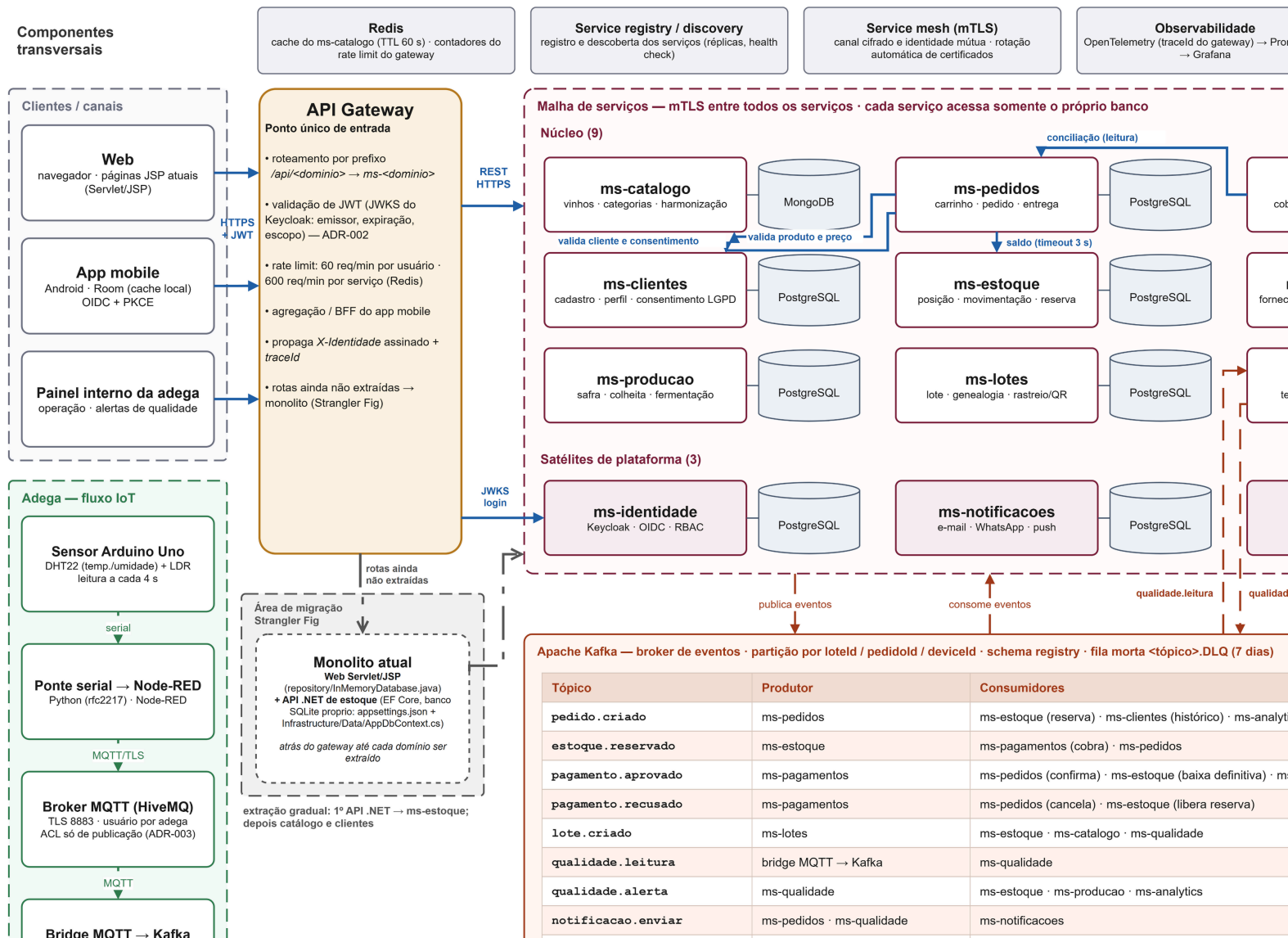


Figura 1 — Arquitetura de microsserviços da Vinheria Agnello: visão geral com os 12 serviços, o API Gateway, o Kafka, a migração Strangler Fig e o fluxo IoT da adega

As figuras deste documento usam os arquivos `.png` (3992 × 3126 px, cerca de 394 dpi na largura útil de uma página A4 paisagem); os `.svg` do mesmo desenho, com o texto convertido em curvas, estão no repositório, em `Documentacao/Fase6/`.

2.1 Como ler o diagrama

O desenho é lido da esquerda para a direita. Os **clientes** — web (páginas JSP atuais, em `Web/src/main/java/br/com/fiap/vinheriaagnello/`), app mobile (Android/Room, em `Mobile/app/src/main/java/com/example/myapplication/data/local/`) e painel interno da adega — só entram no sistema pelo **API Gateway**. Ele roteia por prefixo (`/api/<dominio> → ms-<dominio>`), valida o JWT emitido pelo `ms-identidade` (Keycloak, ADR-002 — `Documentacao/Fase6/adr/ADR-002-validacao-token-gateway.md`), aplica o *rate limit* com contadores no Redis e faz a agregação (BFF) do app mobile. Dentro da **malha de serviços**, toda chamada trafega com mTLS.

Os **12 serviços** da lista congelada da seção 1 aparecem com os mesmos nomes: 9 de núcleo e 3 satélites de plataforma. Cada um tem **seu próprio banco** (Database per Service): PostgreSQL na maioria, MongoDB no `ms-catalogo`, TimescaleDB (série temporal) no `ms-qualidade` e um read-model PostgreSQL no `ms-analytics`. Nenhuma linha liga um serviço ao banco de outro. As setas azuis contínuas dentro da malha são as chamadas **síncronas** entre serviços: `ms-pedidos → ms-estoque` (consulta de saldo), `ms-pedidos → ms-catalogo` e `ms-pedidos → ms-clientes` (validação na criação do pedido) e `ms-pagamentos → ms-pedidos` (consulta administrativa de conciliação). Elas existem onde a resposta é necessária para continuar; todo o resto da conversa entre serviços é evento no Kafka.

A camada assíncrona é o **Apache Kafka**. A tabela dentro do bloco lista cada tópico com seu produtor e seus consumidores. O fluxo de compra segue a saga da seção 5: `pedido.criado → estoque.reservado → pagamento.aprovado` ou `pagamento.recusado → confirmação` ou `cancelamento → notificacao.enviar`.

O **fluxo IoT** aparece à esquerda, embaixo: sensor Arduino (`Arduino/VinheriaSensores/VinheriaSensores.ino`) → ponte serial (`MQTT/bridge_wokwi_nodered.py`) → Node-RED (`MQTT/node_red_vinheria_flow.json`) → broker MQTT (TLS, usuário por adega, ADR-003 — `Documentacao/Fase6/adr/ADR-003-mqtt-identidade-por-dispositivo.md`) → *bridge* MQTT → Kafka, que publica `qualidade.leitura`. O `ms-qualidade` consome a leitura e publica `qualidade.alerta` e `notificacao.enviar`.

A **área de migração** mostra o *Strangler Fig*: o monolito atual — o web Servlet/JSP (`Web/src/main/java/br/com/fiap/vinheriaagnello/`, com `repository/InMemoryDatabase.java`) e a API .NET de estoque (`Web/src/VinheriaAgnello.Server/`, EF Core com um único banco SQLite configurado em `src/VinheriaAgnello.API/appsettings.json` e mapeado em `src/VinheriaAgnello.Infrastructure/Data/AppDbContext.cs`) — fica atrás do gateway, que continua mandando para ele as rotas ainda não extraídas. O primeiro domínio extraído é o estoque, porque a API .NET já é o embrião do `ms-estoque` (seção 1.4). Catálogo e clientes vêm em seguida.

2.2 Legenda

Tabela 7 — Legenda dos traços e dos tipos de comunicação do diagrama

Traço	Significado
Linha contínua grossa, azul	Síncrono — REST/HTTPS (dentro da malha com mTLS)
Linha tracejada longa, vermelho-tijolo	Assíncrono — evento publicado/consumido no Kafka
Linha pontilhada, verde	Telemetria IoT — serial e MQTT sobre TLS
Traço-ponto, cinza, seta aberta	Migração Strangler Fig (monolito → serviço)
Linha fina sem seta	Serviço ↔ banco próprio (acesso exclusivo)

Os tipos de comunicação se distinguem pelo **padrão do traço**, não só pela cor, então a figura continua legível em impressão preto e branco.

2.3 Conferência dos critérios de aceite

Tabela 8 — Conferência dos critérios do item 2

Critério	Situação
Todo microserviço da seção 1 aparece; nenhum serviço inventado	12 de 12, com os nomes do glossário. Redis, registry, mesh, observabilidade, Vault e a <i>bridge</i> MQTT → Kafka são infraestrutura, não serviços de domínio (mesmo critério da seção 1.3, para o Node-RED)
Síncrono × assíncrono distinguíveis pela legenda	Sim: traço contínuo × tracejado, além da cor — legenda repetida em cada uma das 4 páginas
Nenhum acesso direto ao banco de outro serviço; cliente só via gateway	Sim: cada banco liga a um único serviço; os três clientes apontam apenas para o gateway
Nomes dos tópicos iguais aos do texto	Os 9 tópicos da seção 5 (<code>pedido.criado</code> , <code>estoque.reservado</code> , <code>pagamento.aprovado</code> , <code>pagamento.recusado</code> , <code>lote.criado</code> , <code>qualidade.leitura</code> , <code>qualidade.alerta</code> , <code>notificacao.enviar</code> , <code>cliente.anonimizado</code>) + <code>recebimento.confirmado</code> , marcado como proposto
Arquivos entregues (.drawio + .png e .svg)	Sim: um <code>.drawio</code> com 4 páginas e 4 conjuntos <code>.png/.svg</code> . Os nomes seguem o <code>README.md</code> desta pasta (<code>P2_arquitetura.*</code>); o card da atividade citava <code>fase6_arquitetura.*</code> — ver seção 2.5
Legível impresso (fonte ≥ 12 no tamanho final)	Atendido pelas figuras de detalhe : 12,2 a 12,7 pt em A4 paisagem com margens de 2 cm. A visão geral não atende sozinha — nela a menor fonte imprime a ~3,7 pt e os nomes dos serviços a ~4,5 pt — e serve para leitura em tela/zoom, como o SVG permite. Medições na seção 2.4

2.4 Figuras de detalhe e tamanho de impressão

A figura única tem 1976 × 1543 unidades de desenho e cobre o sistema inteiro: nesse tamanho, para imprimir tudo numa A4 paisagem (25,7 cm × 17 cm úteis) o desenho é reduzido a ~43%, e nem a maior fonte da figura chega a 12 pt. Para cumprir o critério **sem** encolher o desenho nem perder informação, o `.drawio` passou a ter quatro páginas, e as três de detalhe foram desenhadas com fonte mínima de 24 px num canvas estreito (≈ 1400 px):

Tabela 9 — Páginas do diagrama, tamanho de desenho e legibilidade de impressão

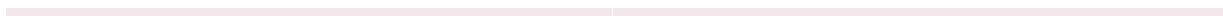
Página	Conteúdo	Desenho	Menor fonte no PDF exportado	Impresso em A4 paisagem (margens de 2 cm)
1 — Visão geral	sistema completo (<code>P2_arquitetura.png</code> / <code>.svg</code>)	1976 × 1543	8,6 pt (notas e rótulos de seta)	~3,7 pt (nomes de serviço: ~4,5 pt) — leitura em tela/zoom
2 — Detalhe 1	canais → gateway → 12 serviços, cada um com seu banco (<code>P2_arquitetura_d1_servicos.*</code>)	1371 × 900	17,3 pt	~12,7 pt
3 — Detalhe 2	Kafka: 10 tópicos, produtores e consumidores (<code>P2_arquitetura_d2_eventos.*</code>)	1366 × 945	17,3 pt	~12,2 pt
4 — Detalhe 3	fluxo IoT ponta a ponta + migração Strangler Fig (<code>P2_arquitetura_d3_iot_migracao.*</code>)	1397 × 925	17,3 pt	~12,4 pt

Como foi medido (não é estimativa): o `.drawio` foi exportado para PDF e a menor fonte de cada

página foi lida do próprio PDF; o valor impresso é essa fonte multiplicada pelo fator de escala necessário para caber na área útil de uma A4 paisagem (0,70–0,74 nas páginas de detalhe, 0,43 na visão geral). Como referência, uma fonte de 24 px no draw.io equivale a 18 pt com o desenho em tamanho natural.

As três páginas de detalhe entram em páginas paisagem deste documento (Figuras 2 a 4); em A4 retrato as mesmas figuras cairiam para ~8,4 pt. A visão geral continua sendo a figura de abertura da seção (em tela, o SVG permite zoom).

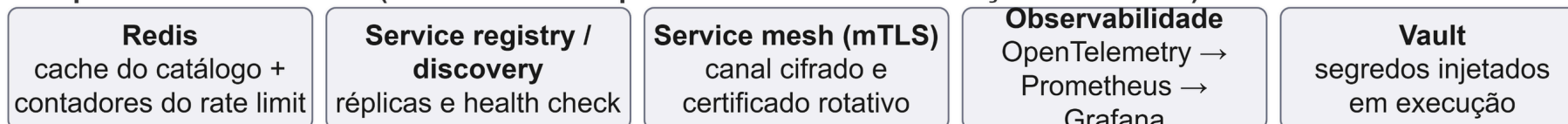
As figuras de detalhe não acrescentam conteúdo novo ao sistema desenhado: repetem, em escala legível, o que já está na visão geral, e cada uma traz a própria legenda, para poder ser lida isolada.



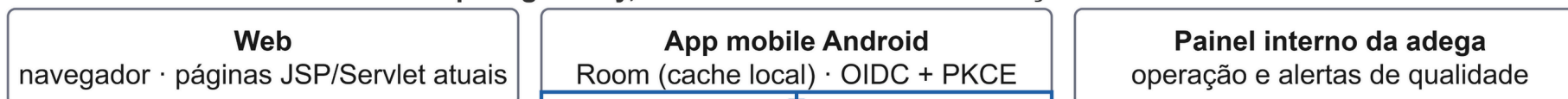
Vinheria Agnello — Arquitetura de microsserviços · detalhe 1/3

Canais · API Gateway (ponto único de entrada) · 12 serviços da lista congelada do P1, cada um com o próprio banco

Componentes transversais (infraestrutura de plataforma — não são serviços de domínio)



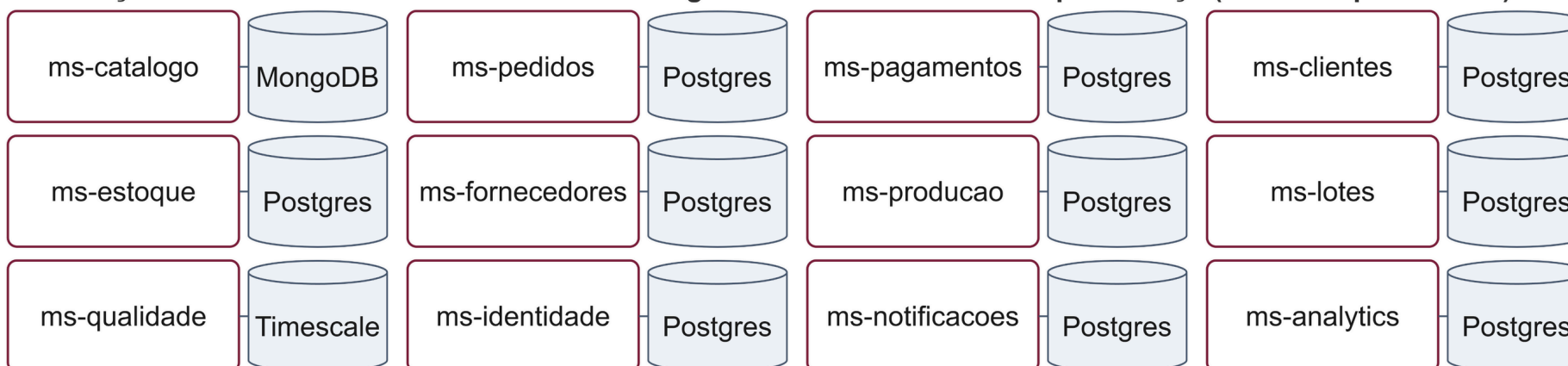
Clientes / canais — todos entram pelo gateway, nenhum fala direto com serviço



API Gateway — ponto único de entrada

- roteia /api/<dominio> → ms-<dominio> (mTLS dentro da malha)
- valida JWT (JWKS do Keycloak: emissor, expiração, escopo) — ADR-002
- rate limit: 60 req/min por usuário e 600 req/min por serviço (Redis)
- agregação / BFF do app mobile
- rotas não extraídas → monolito (Strangler Fig)

12 serviços — 9 de núcleo + 3 satélites · nomes do glossário do P1 · um banco por serviço (Database per Service)



→ Síncrono — REST/HTTPS (dentro da malha com mTLS) — Serviço ↔ banco próprio (acesso exclusivo)

Figura 2 — Detalhe 1: canais, API Gateway e os 12 serviços, cada um com o seu próprio banco

Vinheria Agnello — Arquitetura de microsserviços · detalhe 2/3

Apache Kafka — tópicos, produtores e consumidores · nomes iguais aos da lista do P5 (o texto do P4 usa esta tabela)

partição por loteld / pedidold / deviceld · schema registry (evolução de esquema compatível) · fila morta <topico>.DLQ com retenção de 7 dias e alerta

Tópico	Produtor	Consumidores
<code>pedido.criado</code>	ms-pedidos	ms-estoque (reserva) · ms-clientes (histórico) · ms-analytics
<code>estoque.reservado</code>	ms-estoque	ms-pagamentos (cobra) · ms-pedidos
<code>pagamento.aprovado</code>	ms-pagamentos	ms-pedidos (confirma) · ms-estoque (baixa) · ms-analytics
<code>pagamento.recusado</code>	ms-pagamentos	ms-pedidos (cancela) · ms-estoque (libera reserva)
<code>lote.criado</code>	ms-lotes	ms-estoque · ms-catalogo · ms-qualidade
<code>qualidade.leitura</code>	bridge MQTT → Kafka	ms-qualidade
<code>qualidade.alerta</code>	ms-qualidade	ms-estoque · ms-producao · ms-analytics
<code>notificacao.enviar</code>	ms-pedidos · ms-qualidade	ms-notificacoes
<code>cliente.anonimizado</code>	ms-clientes	ms-pedidos · ms-notificacoes · ms-analytics
<code>recebimento.confirmado</code> *	ms-fornecedores	ms-estoque (registra a entrada)

* recebimento.confirmado: proposto no P1 (item 7); o nome é confirmado na parte 4. ms-analytics também assina os demais tópicos de negócio para montar o read-model.

→ Síncrono — REST/HTTPS (dentro da malha com mTLS) — — ➤ Assíncrono — evento publicado/consumido no Kafka

Figura 3 — Detalhe 2: tópicos do Kafka, produtores e consumidores

Vinheria Agnello — Arquitetura de microsserviços · detalhe 3/3

Adega: sensor → MQTT → bridge → Kafka → ms-qualidade → alerta, e a migração do monolito (Strangler Fig)

Sensor Arduino Uno — DHT22 (temperatura e umidade) + LDR · leitura a cada 4 s

serial (9600 baud)

Ponte serial → **Node-RED** — Python (RFC 2217) e fluxo Node-RED

HTTP POST /vinheria/sensores

Broker MQTT HiveMQ — TLS 8883 · usuário por adega, ACL só de publicação (ADR-003)

tópico Fiap/iot/3ESOA/CodeSquad/sensor

Bridge MQTT → **Kafka** — traduz o tópico legado Fiap/iot/3ESOA/CodeSquad/sensor

publica qualidade.leitura

ms-qualidade — consome qualidade.leitura e publica qualidade.alerta

evento qualidade.alerta

ms-notificacoes + painel da adega — alerta chega a quem opera a adega

Área de migração — Strangler Fig

O monolito atual fica atrás do gateway até cada domínio ser extraído:

- Web Servlet/JSP — *Web/src/main/java/br/com/fiap/vinheriaagnello/*
- API .NET de estoque — *Web/src/VinheriaAgnello.Server/*
- extração gradual: 1º a API .NET → ms-estoque; depois catálogo e clientes

Regras do desenho

- cliente só fala com serviço através do API Gateway
- nenhum serviço acessa o banco de outro — dado cruzado só por API ou evento
- nomes de serviços e de tópicos = glossário do P1 + lista do P5
- infraestrutura (Redis, registry, mesh, observabilidade, Vault, Node-RED, bridge) não é serviço de domínio

Legenda

- Síncrono — REST/HTTPS (mTLS)
- Assíncrono — evento no Kafka
- ...→ Telemetria IoT — serial e MQTT sobre TLS
- Migração Strangler Fig (monolito → serviço)

Blocos do fluxo IoT: as etapas são encadeadas na ordem de leitura (topo → base).

Figura 4 — Detalhe 3: fluxo IoT da adega (sensor → MQTT → bridge → Kafka) e migração do monolito

2.5 Pontos a confirmar com o grupo

- **recebimento.confirmando** — aparece no diagrama com asterisco, como proposta da seção 1.7. Se a seção 4 mudar o nome, ajustar a linha da tabela no `.drawio` e reexportar.
- **Produtores e consumidores por tópico** — a tabela é a proposta desta parte. Principais escolhas: o `ms-notificacoes` só dispara comunicação a partir de `notificacao.enviar` (contrato de entrada único) — o `cliente.anonimizado` que ele também assina serve apenas para limpar preferência de canal e não gera envio; e `qualidade.alerta` vai para estoque, produção e analytics. O Arthur (seção 4) deve usar a mesma tabela em "quem fala com quem"; a conferência linha a linha foi feita em 01/10/2026 e as divergências encontradas estão registradas nos itens abaixo.
- **Nome dos arquivos** — o card da atividade no board do grupo pedia `fase6_arquitetura.*`; o repositório usa `P2_arquitetura.*`, no padrão das outras partes (`P1_servicos.md`, `P3_padroes.md`, `P4_integracao.md`) e da tabela do `README.md` desta pasta. Mantivemos o padrão do repositório, para a consolidação não ter dois nomes para o mesmo item; se o professor exigir o nome literal do card, renomear é uma troca de arquivo, sem impacto no conteúdo.
- **Revisão cruzada com André (seção 3) e Arthur (seção 4)** — combinada para 26–28/09 e concluída em 01/10/2026: cada padrão justificado na seção 3 existe no desenho, e a tabela de tópicos da seção 4 foi conferida linha a linha contra a página 3 deste diagrama.
- **lote.criado** — produtor `ms-lotes` e consumidores `ms-estoque`, `ms-catalogo` e `ms-qualidade` (mais `ms-analytics`, pelo rodapé da tabela), conforme a posse de Lote, Genealogia e QR na seção 1.1.4. A seção 4 foi alinhada a esta linha na revisão de 01/10/2026; a frase do card que dizia "o ms-producao cria o lote" fica superada por esta decisão, e o `ms-producao` segue dono da safra e do ciclo produtivo.
- **cliente.anonimizado** — consumidores `ms-pedidos`, `ms-notificacoes` e `ms-analytics`, como na tabela, e não só os dois últimos. No `ms-notificacoes` o evento apenas limpa preferência de canal: quem dispara comunicação continua sendo `notificacao.enviar`, conforme a correção do item anterior.
- **Chamadas síncronas internas** — resolvido em 01/10/2026: a página 1 passou a desenhar as quatro chamadas entre serviços que a matriz da seção 4 lista (`ms-pedidos → ms-estoque`, `ms-pedidos → ms-catalogo`, `ms-pedidos → ms-clientes` e `ms-pagamentos → ms-pedidos`), em setas azuis contínuas com os rótulos "saldo (timeout 3 s)", "valida produto e preço", "valida cliente e consentimento" e "conciliação (leitura)". Os prazos de cada chamada são os da seção 5.6.

3. Padrões de Arquitetura e Justificativas

Responsável: André Luiz dos Santos Flores — RM 554952 · Período: 22/09/2026 a 30/09/2026 · Versão 1.1, com revisão de consolidação de 30/09/2026

3.1 Padrões obrigatórios

3.1.1 API Gateway Pattern

O que resolve → O API Gateway cria um ponto único de entrada para os clientes e evita que Web, aplicativo mobile e painel interno precisem conhecer diretamente o endereço e a topologia de cada microsserviço. Além do roteamento, ele concentra responsabilidades transversais da borda da aplicação, como validação de autenticação, limitação de requisições e composição de respostas para diferentes canais.

Como aparece no nosso sistema → Na arquitetura da Vinheria Agnello, todos os canais entram obrigatoriamente pelo **API Gateway**. Ele roteia requisições no formato `/api/<dominio>` para o respectivo `ms-<dominio>`, valida o JWT emitido pelo `ms-identidade`, aplica *rate limit* com contadores no Redis e realiza agregação/BFF para o aplicativo mobile. Assim, uma operação do Web ou do app pode chegar, por exemplo, ao `ms-catalogo`, `ms-pedidos` ou `ms-clientes` sem que o cliente acesse diretamente esses serviços. Durante a migração, as rotas ainda não extraídas continuam sendo encaminhadas pelo gateway ao sistema legado.

Trade-off/limitação → O gateway passa a ser um componente crítico da arquitetura. Uma falha ou configuração incorreta pode afetar o acesso a vários serviços ao mesmo tempo. Ele também precisa ser escalável e monitorado para não se tornar gargalo nem concentrar regras de negócio que deveriam permanecer dentro dos microsserviços.

3.1.2 Service Discovery Pattern

O que resolve → O Service Discovery permite que os componentes da arquitetura localizem instâncias dos serviços sem depender de endereços IP fixos. Isso é importante em um ambiente distribuído, porque instâncias podem ser reiniciadas, substituídas ou replicadas sem exigir reconfiguração manual dos consumidores.

Como aparece no nosso sistema → A seção 2 representa explicitamente um componente de **Service registry / discovery**, responsável por registrar e localizar réplicas e por disponibilizar informações de *health check*. Dessa forma, serviços como `ms-pedidos`, `ms-estoque` e `ms-pagamentos` podem ser localizados pela plataforma sem acoplamento a um endereço fixo. A escolha aqui é pelo **ambiente**, não por moda: o alvo do grupo é uma plataforma orquestrada, como o próprio seção 5 assume ao dimensionar por *pod*, com HPA e mínimo de 2 réplicas nos serviços de núcleo (seção 5.5). Nesse ambiente a descoberta é nativa — cada serviço é exposto por um `Service` com DNS interno, e as réplicas entram e saem do balanceamento conforme o *readiness probe*, sem catálogo externo de endpoints. Consul ou Eureka se justificariam **fora** de um orquestrador, onde esse catálogo não existe: mantê-los aqui acrescentaria um segundo plano de controle para resolver o que o DNS interno já resolve. O padrão permanece o mesmo — muda só quem executa o papel de registry/discovery.

Trade-off/limitação → A descoberta de serviços adiciona infraestrutura e operação ao ambiente. Se o registry estiver indisponível ou com informações desatualizadas, novas chamadas podem não localizar uma instância saudável. Por isso, esse componente também precisa de alta disponibilidade e monitoramento. Delegar a descoberta à plataforma transfere essa responsabilidade para ela: `Service`, DNS interno e *probes* passam a ser parte do caminho crítico, e um *readiness probe* mal configurado tira do balanceamento uma instância que está no ar.

3.1.3 Database per Service Pattern

O que resolve → O Database per Service define que cada microsserviço é o único dono dos seus dados e que nenhum serviço acessa diretamente as tabelas ou coleções de outro. Isso reduz o acoplamento entre domínios e permite que cada serviço evolua seu modelo de dados de acordo com sua própria responsabilidade.

Como aparece no nosso sistema → Os 12 serviços da Vinheria possuem bancos próprios no diagrama. O `ms-catalogo` usa MongoDB, o `ms-qualidade` usa TimescaleDB e os demais serviços usam majoritariamente PostgreSQL, incluindo o read-model do `ms-analytics`. A posse de dados também foi definida na seção 1: o `ms-estoque`, por exemplo, é o único dono de Posição de Estoque, `MovimentacaoEstoque` e Reserva; o `ms-pedidos` é dono de Carrinho, Pedido, Item do Pedido e Status de Separação/Entrega; e o `ms-pagamentos` é dono de Cobrança, Transação, Conciliação e Estorno. Quando um domínio precisa de informação de outro, a troca deve acontecer por API ou evento, e não por acesso direto ao banco vizinho.

No sistema atual existem formas diferentes de persistência que serão gradualmente substituídas ou reposicionadas durante a migração: o Web Servlet/JSP utiliza `Web/src/main/java/br/com/fiap/vinheriaagnello/repository/InMemoryDatabase.java`, a API .NET de estoque possui seu próprio SQLite configurado em `Web/src/VinheriaAgnello.Server/src/VinheriaAgnello.API/appsettings.json` e mapeado por `Web/src/VinheriaAgnello.Server/src/VinheriaAgnello.Infrastructure/Data/AppDbContext.cs`, e o aplicativo Android utiliza Room em `Mobile/app/src/main/java/com/example/myapplication/data/local/`.

Trade-off/limitação → Consultas que antes poderiam ser resolvidas diretamente em um banco deixam de poder fazer `JOIN` entre domínios. Dados cruzados passam a ser obtidos por APIs, eventos ou projeções de leitura, como ocorre no `ms-analytics`. Isso aumenta a complexidade da integração e pode introduzir consistência eventual entre diferentes serviços.

3.1.4 Saga Pattern

O que resolve → O Saga Pattern coordena uma operação de negócio que atravessa vários microsserviços sem depender de uma única transação distribuída. Cada serviço executa sua própria transação local e, quando alguma etapa falha, o fluxo precisa executar ações compensatórias para desfazer ou neutralizar o que já havia sido realizado.

Como aparece no nosso sistema → O fluxo de compra da Vinheria utiliza eventos no Apache Kafka. O `ms-pedidos` publica `pedido.criado`; o `ms-estoque` consome esse evento, realiza a reserva e publica `estoque.reservado`; o `ms-pagamentos` processa a cobrança e publica `pagamento.aprovado` ou `pagamento.recusado`. Em caso de aprovação, o pedido segue para confirmação e baixa definitiva do estoque. Em caso de recusa, o pedido é cancelado e a reserva precisa ser liberada, funcionando como compensação da etapa já executada.

3.1.4.1 Coreografia x orquestração

Na **orquestração**, existiria um coordenador central responsável por determinar a próxima etapa da saga. Isso facilita acompanhar o fluxo em um único ponto, mas cria um componente adicional que conhece a sequência completa da operação.

Na **coreografia**, cada serviço reage aos eventos publicados pelos demais. O fluxo fica mais desacoplado, porque os participantes não dependem de um coordenador central, porém o comportamento completo fica distribuído entre vários produtores e consumidores de eventos.

3.1.4.2 Escolha para a Vinheria

Para a arquitetura desenhada, a escolha é **Saga por coreografia**. A seção 2 representa o fluxo de

compra como uma sequência de eventos no Kafka entre `ms-pedidos`, `ms-estoque` e `ms-pagamentos`, e não apresenta um serviço separado atuando como orquestrador. A tabela de tópicos também mostra cada participante publicando o resultado de sua etapa para que o próximo consumidor reaja.

Trade-off/limitação → A coreografia reduz o acoplamento direto, mas torna mais difícil enxergar e depurar o fluxo completo, porque o estado da operação fica distribuído entre vários serviços e eventos. Também exige atenção às compensações e às situações em que um consumidor esteja temporariamente indisponível.

3.2 Padrões adicionais

Os padrões abaixo foram escolhidos entre os extras sugeridos na atividade porque aparecem de forma explícita na arquitetura da seção 2. Outros padrões possíveis não foram incluídos apenas por quantidade, para manter a entrega consistente com o diagrama.

3.2.1 Strangler Fig Pattern

O que resolve → O Strangler Fig permite migrar gradualmente um sistema legado para uma nova arquitetura, sem exigir que todo o monolito seja substituído de uma única vez. Novos domínios passam a ser atendidos pelos microsserviços enquanto o restante continua temporariamente no sistema atual.

Como aparece no nosso sistema → O diagrama mantém o sistema atual atrás do API Gateway. As rotas ainda não extraídas continuam indo para o Web Servlet/JSP em `Web/src/main/java/br/com/fiap/vinheriaagnello/` e para a API .NET em `Web/src/VinheriaAgnello.Server/`. A extração planejada começa pela API .NET de estoque, que serve como embrião do `ms-estoque`, e depois segue para catálogo e clientes, atendidos futuramente por `ms-catalogo` e `ms-clientes`.

Trade-off/limitação → Durante a transição, a equipe precisa manter temporariamente o legado e os novos microsserviços ao mesmo tempo. Isso aumenta a complexidade de roteamento, testes, observabilidade e manutenção até que todos os domínios previstos sejam extraídos.

3.2.2 Cache (Redis) + API Composition

O que resolve → O uso de cache reduz acessos repetidos a dados que podem ser reutilizados por um período curto, enquanto API Composition permite combinar respostas de mais de um serviço para atender um cliente sem obrigá-lo a realizar várias chamadas diretamente.

Como aparece no nosso sistema → O Redis aparece como componente transversal da arquitetura. Na seção 2, ele é usado como cache do `ms-catalogo`, com TTL de 60 segundos, e também pelos contadores de *rate limit* do API Gateway. O gateway realiza agregação/BFF para o aplicativo mobile, compondo respostas sem expor diretamente a malha interna de serviços ao cliente.

Trade-off/limitação → O cache pode entregar dados temporariamente desatualizados até o vencimento ou a invalidação da entrada. A composição de respostas no gateway também aumenta a responsabilidade desse componente e pode elevar o tempo de resposta quando depende de vários serviços simultaneamente.

3.2.3 Service Mesh com mTLS

O que resolve → O Service Mesh centraliza preocupações da comunicação entre serviços, principalmente segurança do tráfego interno. O uso de mTLS permite que as duas pontas da comunicação se autenticuem mutuamente e que os dados trafeguem cifrados dentro da malha.

Como aparece no nosso sistema → A seção 2 representa um **Service mesh (mTLS)** como componente transversal e informa que toda chamada interna da malha utiliza mTLS, com canal

cifrado e rotação automática de certificados. Assim, comunicações entre serviços como `ms-pedidos`, `ms-estoque`, `ms-pagamentos`, `ms-clientes` e os demais participantes não dependem apenas da proteção aplicada na borda pelo API Gateway. A política de certificados e de identidade entre serviços é detalhada na seção 5.3.4; neste item o *service mesh* é justificado como decisão de arquitetura, sem repetir o texto daquela seção.

Trade-off/limitação → O service mesh adiciona complexidade operacional à plataforma, pois exige gerenciamento dos componentes da malha, certificados, políticas e observabilidade. Também adiciona uma camada extra ao caminho de comunicação, que precisa ser corretamente configurada e monitorada para não dificultar o diagnóstico de falhas.

3.3 Padrões avaliados e não incluídos nesta versão

A atividade sugere outros padrões que poderiam fortalecer a arquitetura, como Circuit Breaker, Retry com backoff, Bulkhead, Transactional Outbox, Idempotent Consumer e CQRS. Eles não entram nesta lista por dois motivos distintos, que não devem ser confundidos:

- **Circuit Breaker, Retry com backoff, Bulkhead, dead letter queue e idempotência no consumidor estão na arquitetura** — são tratados como **políticas de resiliência** na seção 5.6, com parâmetros definidos (janela de erro, número de tentativas, retenção da DLQ). Aqui não são repetidos para não duplicar texto entre as seções do mesmo documento: a seção 3 justifica o padrão, a seção 5 detalha a política.
- **CQRS e Transactional Outbox continuam fora** porque não existem no desenho da seção 2: não há read model de catálogo nem tabela de *outbox* representados. Se entrarem na seção 2 antes da consolidação, esta seção é revista; caso contrário, citá-los aqui seria justificar um padrão que o diagrama não sustenta.

O critério que vale para os **padrões de arquitetura** desta entrega continua sendo o do diagrama: só entra o que tem representação explícita ou suporte direto na seção 2. O que muda na redação é não tratar os itens acima como se estivessem fora da arquitetura — os de resiliência estão nela, descritos na parte da seção 5 que é dona desse assunto.

3.4 Conferência dos critérios de aceite

Tabela 10 — Conferência dos critérios do item 3

Critério do item 3 da atividade	Situação
Os 4 padrões obrigatórios têm justificativa com exemplo do sistema	Atendido: API Gateway, Service Discovery, Database per Service e Saga
Cada padrão cita ao menos um serviço pelo nome do diagrama	Atendido nos blocos correspondentes
Todo padrão apresenta trade-off ou limitação	Atendido em todos os 7 padrões
Saga compara coreografia e orquestração	Atendido na seção 3.1.4
Saga escolhe uma abordagem com argumento	Atendido: coreografia, por ser a abordagem representada na seção 2
Padrões extras aparecem no diagrama	Atendido: Strangler Fig, Redis/API Composition e Service Mesh com mTLS
Padrões de resiliência não duplicados entre as seções 3 e 5	Atendido: Circuit Breaker, Retry, Bulkhead, DLQ e idempotência ficam como política na seção 5.6 (seção 3.3)
Nomes dos serviços seguem o glossário congelado da seção 1	Atendido
Nenhum serviço novo foi inventado	Atendido

3.5 Dependências e fronteiras com as outras partes

- **seção 1 (Roger):** fornece a lista congelada de serviços, responsabilidades e posse de dados

usada principalmente no Database per Service.

- **seção 2 (Kevin):** fornece o desenho que valida quais padrões realmente aparecem na arquitetura.
- **seção 4 (Arthur):** detalha a comunicação entre serviços e o fluxo de mensagens. Nesta seção 3, a Saga é apresentada apenas como padrão arquitetural e justificada pelo seu propósito.
- **seção 5 (Yasmin):** reutiliza estas decisões na seção de governança, segurança, tolerância a falhas e consolidação. Fronteira acordada: a seção 3 justifica o padrão (o "porquê"); as políticas de resiliência, com parâmetros, ficam na seção 5.6 (o "como"), para não haver texto repetido entre as duas seções.

3.6 Pontos a confirmar com o grupo

- **Saga por coreografia:** esta entrega assume coreografia porque a seção 2 mostra os serviços reagindo a eventos no Kafka e não apresenta um orquestrador separado.
 - **Service Discovery:** a escolha feita na seção 3.1.2 assume plataforma orquestrada (descoberta nativa por `Service` + DNS interno), pelo ambiente que a seção 5 já adota. A seção 2 desenha o registry/discovery sem nomear a tecnologia; se o grupo preferir declarar Consul ou Eureka, ou nomear a plataforma explicitamente na seção 5, o parágrafo da seção 3.1.2 muda em uma linha.
 - **Padrões adicionais futuros:** Outbox e CQRS só devem entrar na seção 3 se forem também representados na seção 2, para preservar a consistência entre texto e diagrama. Os padrões de resiliência (Circuit Breaker, Retry, Bulkhead, DLQ, idempotência) permanecem na seção 5, para o documento não repetir a mesma explicação em duas seções.
-

4. Integração entre Serviços: Comunicação Síncrona x Assíncrona

Responsável: Arthur Cei Corrêa — RM 97781 · Período: 29/09/2026 a 05/10/2026 · Versão 1.4, com revisão de consolidação de 01/10/2026

Este item define **como** os serviços se integram. A seção 3 explica **por que** a compra usa Saga por coreografia; aqui estão os canais, contratos e decisões concretas dessa Saga.

4.1 Regra de decisão

Usamos **síncrono (REST/HTTPS com JSON)** quando o usuário está esperando e a resposta é necessária para continuar: consulta de catálogo, consulta de saldo de estoque, validação de cliente, criação do pedido, autorização no gateway e consulta pública de rastreio. O gateway é a única porta dos clientes; entre serviços, o tráfego usa HTTPS com mTLS na malha.

Usamos **assíncrono (Kafka)** quando o efeito pode acontecer depois, precisa absorver picos ou vários interessados precisam saber do fato: reserva e liberação de estoque, resultado do pagamento, criação de lote, telemetria, alerta de qualidade, notificação e anonimização. O produtor executa sua transação local e tenta publicar o evento com confirmação de entrega; os consumidores processam em seu próprio ritmo.

O custo do síncrono é o **acoplamento temporal**: uma cadeia de chamadas pode produzir falha em cascata. Toda chamada tem timeout, retry apenas quando idempotente e Circuit Breaker, conforme a seção 5; o gateway não transforma uma indisponibilidade de um serviço em espera infinita.

O custo do assíncrono é a **consistência eventual**. Kafka trabalha com entrega pelo menos uma vez: mensagens podem ser duplicadas, chegar atrasadas ou, entre partições, fora de ordem. Por isso cada evento tem `idEvento`, cada consumidor guarda os IDs já aplicados, a chave de partição preserva a ordem por `pedidoId`, `loteId` ou `deviceId`, e mensagens que excedem as tentativas vão para `<topico>.DLQ`. Lag, taxa de erro e tamanho da DLQ são monitorados.

4.2 Matriz de comunicação

Cada linha representa uma relação direta. A chamada passa pelo API Gateway quando a origem é um canal; chamadas internas usam a malha com mTLS. Os endpoints são versionados conforme a convenção da seção 5.

Tabela 11 — Matriz de comunicação: origem, destino, motivo, tipo e contrato

Origem → destino	Motivo	Tipo	Tópico, se assíncrono	Endpoint, se síncrono
Web/App/Painel → API Gateway	Entrada única, JWT, rate limit e roteamento	Síncrono REST/HTTPS	—	<code>/api/ *</code>
API Gateway → Monolito atual (Web Servlet/JSP + API .NET de estoque)	Rotas cujo domínio ainda não foi extraído continuam sendo encaminhadas ao legado durante a migração (Strangler Fig, conforme a seção 2 e a seção 3)	Síncrono REST/HTTPS	—	Prefixo <code>/api/</code> <code><domínio></code> ainda não extraído

Web/App/Painel → ms-identidade (Keycloak) e API Gateway → ms-identidade (somente JWKS)	Autenticação no fluxo OIDC da seção 5.3.3: Authorization Code + PKCE no app e Client Credentials entre serviços. Quem emite e renova o token é o Keycloak; o gateway apenas busca o JWKS e valida o JWT localmente, sem introspecção por requisição (ADR-002)	Síncrono REST/HTTPS	—	Metadados OIDC/JWKS do Keycloak — a emissão de token não passa pelo gateway
API Gateway → ms-catalogo	Consulta de vinhos e categorias para exibir a oferta	Síncrono REST/HTTPS	—	GET /v1/vinhos
API Gateway → ms-estoque	Consulta de posição disponível no dia a dia	Síncrono REST/HTTPS	—	GET /v1/estoque/posicoes?produtoId=...
API Gateway → ms-pedidos	Criar e consultar pedido	Síncrono REST/HTTPS	—	POST /v1/pedidos ^e GET /v1/pedidos/{pedidoId}
API Gateway → ms-clientes	Consultar ou alterar perfil e consentimento	Síncrono REST/HTTPS	—	GET /v1/clientes/{clienteId}
API Gateway → ms-lotes	Consulta pública do QR Code e genealogia do lote	Síncrono REST/HTTPS	—	GET /v1/lotes/{loteId}/rastreo
ms-pedidos → ms-catalogo	Validar produto, preço vigente e dados do item antes de gravar o pedido	Síncrono REST/HTTPS	—	GET /v1/vinhos/{vinhoId}
ms-pedidos → ms-estoque	Consultar o saldo antes de reservar; é a chamada interna que a seção 2 desenha na malha (rótulo "saldo, timeout 3 s") e o prazo de 3 s é o da seção 5.6.1	Síncrono REST/HTTPS	—	GET /v1/estoque/posicoes?produtoId=...
ms-pedidos → ms-clientes	Validar cliente, endereço e consentimento necessário à venda	Síncrono REST/HTTPS	—	GET /v1/clientes/{clienteId}/validacao-pedido
ms-pedidos → Kafka → ms-estoque , ms-clientes ^e ms-analytics	Solicitar reserva sem bloquear o cliente, atualizar o histórico derivado do cliente e alimentar o read-model analítico	Assíncrono Kafka	pedido.criado	—
ms-estoque → Kafka → ms-pagamentos , ms-pedidos ^e ms-analytics	Liberar a cobrança depois da reserva, atualizar o pedido e alimentar o read-model analítico	Assíncrono Kafka	estoque.reservado	—
ms-pagamentos → Kafka → ms-pedidos , ms-estoque ^e ms-analytics	Informar resultado da cobrança; confirmar, compensar e alimentar o read-model analítico	Assíncrono Kafka	pagamento.aprovado ou pagamento.recusado	—

ms-pedidos → Kafka → ms-notificacoes ^e ms-analytics	Avisar criação, confirmação ou cancelamento e registrar o fato no read-model analítico sem atrasar a resposta do pedido	Assíncrono Kafka	notificacao.enviar	—
ms-pagamentos → ms-pedidos	Consulta administrativa de status e conciliação, quando uma leitura imediata for necessária	Síncrono REST/HTTPS	—	GET /v1/pagamentos/{pedidoId}
ms-lotes → Kafka → ms-estoque, ms-catalogo, ms-qualidade ^e ms-analytics	Criar o registro de rastreabilidade no engarrafamento e avisar quem depende do lote novo (o ms-lotes é dono de Lote, Genealogia e QR pela seção 1.1.4)	Assíncrono Kafka	lote.criado	—
ms-fornecedores → Kafka → ms-estoque ^e ms-analytics	Informar recebimento conferido; o ms-estoque registra a entrada e o analytics atualiza seus indicadores	Assíncrono Kafka	recebimento.confirmando	—
Bridge MQTT → Kafka → ms-qualidade	Transformar telemetria em contrato interno de eventos	Assíncrono Kafka	qualidade.leitura	—
ms-qualidade → Kafka → ms-estoque, ms-producao ^e ms-analytics	Distribuir alerta fora da faixa para ação, operação e histórico	Assíncrono Kafka	qualidade.alerta	—
ms-qualidade → Kafka → ms-notificacoes	Solicitar aviso operacional sobre uma condição crítica	Assíncrono Kafka	notificacao.enviar	—
ms-clientes → Kafka → ms-pedidos, ms-analytics ^e ms-notificacoes	Propagar anonimização sem varredura manual em cópias derivadas; no ms-notificacoes ^o evento só limpa preferência de canal e não dispara envio	Assíncrono Kafka	cliente.anonimizado	—

recebimento.confirmando é mantido porque aparece na seção 2 como tópico proposto para ligar ms-fornecedores a ms-estoque. Caso o grupo remova essa proposta, a linha deve ser removida simultaneamente da seção 2 e deste catálogo.

4.3 Catálogo de eventos e tópicos

Propomos um envelope comum para os eventos: `idEvento`, `tipo`, `versao`, `ocorridoEm`, `traceId`, `chaveParticao` e `payload`. O resumo abaixo descreve o payload de negócio, não o envelope completo.

Tabela 12 — Catálogo de eventos e tópicos

Tópico	Payload resumido	Produtor	Consumidores	Idempotente obrigatório?
pedido.criado	pedidoId, clienteId, itens, totais, endereço resumido, correlationId	ms-pedidos	ms-estoque, ms-clientes, ms-analytics	Sim; uma reserva e uma atualização de histórico por pedidoId
estoque.reservado	pedidoId, reservaId, itens, quantidades e expiração	ms-estoque	ms-pagamentos, ms-pedidos, ms-analytics	Sim; não duplicar cobrança nem transição

<code>pagamento.aprovado</code>	<code>pedidoId</code> , <code>pagamentoId</code> , valor, moeda, data e autorização	<code>ms-pagamentos</code>	<code>ms-pedidos</code> , <code>ms-estoque</code> , <code>ms-analytics</code>	Sim; não confirmar ou baixar duas vezes
<code>pagamento.recusado</code>	<code>pedidoId</code> , <code>pagamentoId</code> , motivo codificado e data	<code>ms-pagamentos</code>	<code>ms-pedidos</code> , <code>ms-estoque</code> , <code>ms-analytics</code>	Sim; liberar a reserva uma única vez
<code>lote.criado</code>	<code>loteId</code> , <code>safraId</code> , origem, data, quantidade e dados de genealogia	<code>ms-lotes</code>	<code>ms-estoque</code> , <code>ms-catalogo</code> , <code>ms-qualidade</code> , <code>ms-analytics</code>	Sim; um QR e uma genealogia por <code>loteId</code>
<code>recebimento.confirmação</code>	<code>recebimentoId</code> , fornecedor, itens, lote(s), quantidades e divergências	<code>ms-fornecedores</code>	<code>ms-estoque</code> , <code>ms-analytics</code>	Sim; não lançar a mesma entrada duas vezes
<code>qualidade.leitura</code>	<code>deviceId</code> , <code>adegaId</code> , <code>loteId</code> opcional, temperatura, umidade, luminosidade e medição	Bridge MQTT	<code>ms-qualidade</code>	Sim por <code>idEvento</code> ; tolerar retry da bridge
<code>qualidade.alerta</code>	<code>alertaId</code> , <code>deviceId</code> , <code>adegaId</code> , tipo, valor, limite, severidade e intervalo	<code>ms-qualidade</code>	<code>ms-estoque</code> , <code>ms-producao</code> , <code>ms-analytics</code>	Sim por <code>alertaId</code> e janela de alerta
<code>notificacao.enviar</code>	<code>notificacaoId</code> , destinatário, canais, template, parâmetros e referência	<code>ms-pedidos</code> ou <code>ms-qualidade</code>	<code>ms-notificacoes</code> , <code>ms-analytics</code>	Sim por <code>notificacaoId</code> ; retry não pode reenviar indevidamente
<code>cliente.anonimizar</code>	<code>clienteId</code> , data, motivo, versão da política e escopos a limpar	<code>ms-clientes</code>	<code>ms-pedidos</code> , <code>ms-analytics</code> , <code>ms-notificacoes</code>	Sim; anonimização é operação monotônica

O consumidor confirma o offset somente depois de concluir a transação local. Um evento duplicado é reconhecido pelo `idEvento` ou pela chave natural do domínio e não reaplica o efeito. Depois do limite de retry, o evento vai para, por exemplo, `pagamento.aprovado.DLQ`; a DLQ tem retenção de sete dias, alerta operacional e procedimento de reprocessamento após correção. Como Transactional Outbox não faz parte das seções 2 e 3, a publicação de cada evento deve usar retry do produtor, confirmação de entrega e alerta de reconciliação para o caso de a transação local ser confirmada e a publicação falhar; a adoção futura de Outbox exigiria atualização do diagrama e da seção 3.

4.4 Fluxos ponta a ponta

4.4.1 Compra de vinho

1. O app envia `POST /v1/pedidos` ao gateway. O gateway valida o JWT e encaminha ao `ms-pedidos`, que consulta o `ms-catalogo` para preço/produto e o `ms-clientes` para a validação do comprador. A resposta síncrona confirma apenas que o pedido foi aceito, não que o pagamento terminou.
2. O `ms-pedidos` grava o pedido como `aguardando_reserva` e publica `pedido.criado`. O `ms-estoque` consome, verifica a posição, cria a Reserva e publica `estoque.reservado`.
3. O `ms-pagamentos` consome `estoque.reservado`, cobra o meio tokenizado e publica `pagamento.aprovado` ou `pagamento.recusado`. Em aprovação, `ms-pedidos` confirma o pedido e publica `notificacao.enviar`; `ms-notificacoes` avisa o cliente e `ms-analytics` registra o fato. Em paralelo, `ms-estoque` consome `pagamento.aprovado` e faz a baixa definitiva. Essa ordem é uma sequência de negócio, não uma cadeia síncrona: consumidores Kafka podem processar em paralelo e não há garantia temporal entre o aviso e a baixa. O `ms-lotes` não participa da

baixa da venda, pois sua rastreabilidade é criada pelo evento `lote.criado`.

4. Em recusa, timeout definitivo ou antifraude negativo, `pagamento.recusado` faz o `ms-estoque` liberar a Reserva e o `ms-pedidos` marcar o pedido como `cancelado`; `ms-notificacoes` informa a falha. Essa é a compensação da Saga: não há transação distribuída e o serviço que criou a reserva é o único que a desfaz.
5. Se uma mensagem for repetida, `pedidoId`, `reservaId` e `pagamentoId` impedem duplicar pedido, reserva, cobrança, baixa ou aviso. Se houver mensagem inválida, o consumidor não bloqueia a partição indefinidamente: envia para a DLQ e gera alerta para reprocessamento.

4.4.2 Alerta de temperatura da adega

O sensor Arduino (`Arduino/VinheriaSensores/VinheriaSensores.ino`) publica a leitura no caminho MQTT existente; a ponte serial e o fluxo Node-RED (`MQTT/node_red_vinheria_flow.json`) levam o dado ao broker e a bridge (`MQTT/bridge_wokwi_nodered.py`) autentica-se nele e traduz a telemetria no evento Kafka `qualidade.leitura`, particionado por `deviceId`. O `ms-qualidade` persiste a série temporal, compara temperatura e umidade com a faixa da adega e, fora do limite, publica `qualidade.alerta` para `ms-estoque`, `ms-producao` e `ms-analytics`. Para avisar pessoas, publica `notificacao.enviar`, consumido pelo `ms-notificacoes`.

O alerta também fica registrado para auditoria com `alertaId`, valor, limite, horário e severidade. Uma leitura duplicada não gera dois alertas porque o consumidor usa `idEvento` e a regra de deduplicação por `alertaId/janela`. Se `ms-notificacoes` estiver indisponível, o Kafka acumula as mensagens; se o processamento falhar repetidamente, `notificacao.enviar.DLQ` recebe a mensagem e o lag dispara um alerta, sem interromper a ingestão do sensor.

4.4.3 Rastreabilidade e consulta do dia a dia

No fluxo produtivo, o `ms-lotes` registra o lote no engarrafamento e publica `lote.criado` — é ele o dono de Lote, Genealogia e QR pela seção 1.1.4, e o `ms-producao` continua dono da safra e do ciclo produtivo, sem publicar esse evento. O `ms-estoque`, o `ms-catalogo` e o `ms-qualidade` consomem o evento para associar posição, ficha e análises ao lote novo, e o `ms-lotes` gera o QR e a genealogia de forma idempotente por `loteId`. Quando o consumidor lê o QR, consulta `GET /v1/lotes/{loteId}/rastreo` no gateway; o endpoint é público e retorna apenas os dados mínimos de origem, safra e histórico.

Separadamente, para a operação diária, o app mobile solicita `GET /v1/estoque/posicoes...` ao gateway. O gateway chama o `ms-estoque`, que responde o saldo atual na hora. Isso é síncrono porque o usuário precisa decidir a venda ou reposição com a posição conhecida; o app pode mostrar o último saldo com aviso de desatualização apenas como degradação controlada, nunca como confirmação de reserva.

4.5 Conferência dos critérios de aceite

Tabela 13 — Conferência dos critérios do item 4

Critério do item 4 da atividade	Situação
Todo par origem → destino da matriz tem tipo de comunicação explícito e justificado	Atendido: 22 linhas, todas com Motivo e com Tipo (síncrono REST/HTTPS ou assíncrono Kafka); a regra de decisão da seção 4.1 explica o critério de escolha
Nomes de tópicos idênticos aos do diagrama	Atendido: os 10 tópicos da seção 2, com a mesma grafia; produtores e consumidores foram conferidos linha a linha contra a página 3 do <code>.drawio</code> na revisão de 01/10/2026
Fluxo de falha (compensação) descrito em pelo menos um fluxo	Atendido: item 4.1, passo 4 (<code>pagamento.recusado</code> libera a Reserva e cancela o pedido), coerente com a seção 5.6.2

Aparece o tratamento de mensagem duplicada (idempotência) e fila morta (DLQ)	Atendido: item 1 (<code>idEvento</code> , chave de partição e <code><topico>.DLQ</code>), coluna "Idempotente obrigatório?" da seção 4.3 e passos 4 e 5 da seção 4.4.1
Fluxos exigidos narrados (compra, qualidade, rastreabilidade, consulta síncrona)	Atendido na seção 4.4, com o caminho de erro da compra e a degradação controlada da consulta de estoque

4.6 Consistência com as seções 1, 2 e 5

- **Serviços:** foram usados somente os 12 nomes congelados no glossário da seção 1.
- **Tópicos:** aparecem os 10 tópicos da seção 2: `pedido.criado`, `estoque.reservado`, `pagamento.aprovado`, `pagamento.recusado`, `lote.criado`, `recebimento.confirmando`, `qualidade.leitura`, `qualidade.alerta`, `notificacao.enviar` e `cliente.anonimizado`. Os consumidores de `pedido.criado` e `notificacao.enviar` também incluem `ms-clientes/ms-analytics`, conforme a seção 2. `recebimento.confirmando` permanece marcado como proposta; se for retirado pelo grupo, deve ser removido dos dois documentos.
- **Checagem de consistência de 01/10/2026:** `lote.criado` passou a ter produtor `ms-lotes` e consumidores `ms-estoque`, `ms-catalogo`, `ms-qualidade` e `ms-analytics`; `cliente.anonimizado` recuperou `ms-pedidos` entre os consumidores. As duas linhas agora são idênticas às da tabela da seção 2, que é a fonte do desenho.
- **Chamadas síncronas internas:** a matriz lista três relações que a seção 2 ainda não desenha (`ms-pedidos` → `ms-catalogo`, `ms-pedidos` → `ms-clientes` e `ms-pagamentos` → `ms-pedidos`), e a seção 2 desenha uma que a matriz não tinha (`ms-pedidos` → `ms-estoque`, "saldo, timeout 3 s") — agora incluída. As três primeiras são chamadas da malha com mTLS e prazo da seção 5.6.1; as três foram desenhadas na página 1 da seção 2 em 01/10/2026, junto da linha `ms-pedidos` → `ms-estoque` que já existia.
- **Resiliência:** timeout, Circuit Breaker, retry restrito a operações idempotentes, DLQ, idempotência por `idEvento` e monitoramento de lag seguem os parâmetros da seção 5.6.
- **Segurança:** clientes entram pelo gateway; chamadas internas usam mTLS; nenhum serviço acessa diretamente o banco de outro serviço.

5. Segurança, Escalabilidade, Resiliência e Governança

Responsável: Yasmin Kimura — RM 557413 · Período: 16/09/2026 a 24/09/2026 · Versão 1.0

5.1 Escopo e ligação com as demais partes

Esta seção responde à segunda metade da atividade da Fase 6: **segurança e governança da arquitetura de microsserviços** — autenticação, autorização, comunicação segura entre serviços, boas práticas de API, gestão de segredos, LGPD, escalabilidade, resiliência, observabilidade e governança de contratos.

Ela não redefine o que já é definido em outras partes. Para não haver divergência de texto (fonte clássica de perda de nota em trabalho em grupo), fica fixada a divisão:

Tabela 14 — Abrangência desta seção e fronteira com as demais

Assunto	Onde é tratado	Papel desta seção
Quais serviços existem, responsabilidade e dono do dado	Seção 1 (seção 1)	Consome os nomes <code>ms-*</code> sem alterá-los
Desenho da arquitetura, gateway, bancos, mensageria	Seção 2 (seção 2)	Consome o desenho; não redesenha
Padrões justificados (Saga, Database per Service etc.)	Seção 3 (seção 3)	Cita o padrão; o "porquê" fica na seção 3
Quem fala com quem, síncrono x assíncrono, tópicos	Seção 4 (seção 4)	Usa exatamente os mesmos nomes de tópicos
Segurança, governança, escala e resiliência	Seção 5 (esta)	Define o "sob quais condições" da arquitetura

Nenhuma decisão aqui altera a lista de serviços nem a matriz de comunicação: ela adiciona as políticas aplicadas sobre eles.

5.2 Retrato do sistema atual (evidências do repositório)

A seção parte de uma auditoria do código que o grupo já entregou nas fases anteriores. Isso importa por dois motivos: (a) mostra que a arquitetura proposta **resolve problemas reais e identificáveis**, e não problemas genéricos de livro; (b) evidencia que o grupo sabe onde está o monolito, o que é pré-requisito da migração via *Strangler Fig*.

Tabela 15 — Retrato do sistema atual: evidência no repositório, risco e limitação

Camada	O que existe hoje	Evidência no repositório	Risco / limitação
Aplicação web (monolito JSP/Servlet)	Login por sessão HTTP; usuários e senhas em <code>ConcurrentHashMap</code> em memória, sem <code>hash</code> ; usuário <code>admin</code> semeado com senha padrão	<code>Web/src/main/java/br/com/fiap/vinheriaagnello/repository/InMemoryDatabase.java</code> (<code>CREDENTIALS.seed()</code>)	Credencial padrão e senha em texto claro; estado em memória não sobrevive a <i>restart</i> nem escala horizontalmente
Filtro de autenticação (web)	<code>@WebFilter(urlPatterns = {"/app/*"})</code> — verifica apenas se existe usuário na sessão	<code>AuthFilter.java</code>	Rotas administrativas <code>ADMIN_WINES = /admin/vinhos</code> e <code>ADMIN_CUSTOMERS = /admin/clientes</code> (<code>WebRoutes.java</code>) ficam fora do filtro : há autenticação, mas não há autorização por papel

API de estoque (.NET)	CORS <code>AllowAll</code> (<code>AllowAnyOrigin</code> + <code>AllowAnyMethod</code> + <code>AllowAnyHeader</code>), <code>UseAuthorization()</code> sem esquema de autenticação registrado, <code>AllowedHosts: "*"</code>	<code>Web/src/VinheriaAgnello.Server/src/VinheriaAgnello.API/Program.cs</code> ; <code>appsettings.json</code>	Qualquer origem consome a API; autorização declarada no <i>pipeline</i> mas sem autenticação efetiva
Banco de dados	SQLite em arquivo único (<code>Data Source=vinheria_agnello.db</code>) com <code>EnsureCreated()</code> no <i>startup</i>	<code>appsettings.json</code> ; <code>Program.cs</code> (<code>lo.db</code>)	Banco único compartilhado por todos os módulos — exatamente o que o padrão <i>Database per Service</i> elimina
Contrato da API	Swagger gerado em tempo de execução, exposto apenas em <code>Development</code> , sem arquivo de contrato versionado	<code>Program.cs</code> (<code>UseSwagger/UseSwaggerUI</code>)	Sem contrato como fonte da verdade: nada impede o <i>mobile</i> e o <i>web</i> divergirem
Mobile (Android)	Banco local Room com entidade <code>Produto</code> (nome, tipo, safra, quantidade, preço) e DAO	<code>Mobile/app/src/main/java/com/example/myapplication/data/local/entity/Produto.kt</code> ; <code>dao/ProdutoDao.kt</code>	Dado em repouso sem criptografia no dispositivo; sincronização com o servidor não é autenticada por identidade de usuário
IoT / adega	Arduino Uno (Wokwi) com DHT22 (temperatura/umidade, pino D2) e LDR (luminosidade bruta, A0), JSON a cada 4000 ms a 9600 baud → ponte Python (<code>rfc2217://localhost:4000</code>) → Node-RED (<code>:1880</code>) → HiveMQ Cloud, 8883 com TLS	<code>Arduino/VinheriaSensores/VinheriaSensores.ino</code> ; <code>MQTT/README_MQTT.md</code>	Credencial MQTT compartilhada com permissão de Publish e Subscribe e tópico <code>Fiap/iot/3ESOA/CodeSquad/sensor</code> : qualquer portador da credencial pode publicar leitura falsa na adega
Dados analíticos	CSV com <code>ID_Venda</code> , <code>Data</code> , <code>Produto</code> , <code>Canal_Venda</code> , <code>Regiao</code> , <code>Valor_Total</code> , <code>Idade_Cliente</code> , <code>Lead_Time_Entrega</code> , <code>Sucesso_Venda</code> + dashboard Tableau (<code>.twbx</code>)	<code>Web/historico_vendas-vinheria_agnello.csv</code> ; <code>Web/vinheria/dashboard.twbx</code>	Idade de cliente em base analítica sem pseudonimização, base legal documentada ou política de retenção

Conclusão da auditoria: o sistema atual tem autenticação frágil (sessão + senha padrão em memória), autorização inexistente por papel, API aberta a qualquer origem, banco único compartilhado, ausência de contrato versionado e credenciais de broker compartilhadas. Todos esses pontos são tratados nas seções 3 a 9 abaixo, **como política da arquitetura de microsserviços** — não como correção pontual do monolito atual (que segue no ar durante a migração).

5.3 Autenticação

5.3.1 Padrão escolhido: OAuth 2.0 / OpenID Connect com Keycloak

- O `ms-identidade` é implementado com **Keycloak** (OIDC), responsável por emissão, renovação e revogação de credenciais. Nenhum serviço de negócio implementa tela de login, armazena senha ou valida senha.
- **Access token JWT de vida curta (5 min) + refresh token rotativo** (7 dias para web, 30 dias

para o app mobile com renovação por uso). Vida curta reduz o dano de um token vazado e torna a revogação eficaz na prática.

- Claims do token: `sub` (identificador do cliente/usuário), `perfis` (papéis — seção 5.4), `escopo`, `cliente_loja` (para operador lotado em uma loja), `exp`, `iat`, `jti`. Os claims **não carregam dado pessoal** (nome, CPF, e-mail) — só identificadores: menos dado circulando em log e em *trace* (LGPD, seção 5.8).

5.3.2 Onde o token é validado: no API Gateway

Decisão explícita (**ADR-002**, seção 5.9.4): o **API Gateway** valida assinatura (JWKS), expiração, emissor, público-alvo e escopo do token. Os microsserviços de negócio **confiam na identidade propagada** e revalidam apenas o mínimo que lhes é próprio (por exemplo, o `ms-pagamentos` revalida a assinatura mesmo tendo passado pelo gateway).

Justificativa (trade-off explícito): validar em todos os serviços é mais seguro e mais custoso — cada serviço passa a depender do Keycloak e a duplicar lógica de token. Centralizar no gateway elimina duplicação e ponto de falha nos serviços, mas exige que a comunicação interna seja fechada (o que é garantido por mTLS, seção 5.3.4). As duas coisas juntas: **validação central na borda + canal interno autenticado**.

5.3.3 Fluxos por tipo de cliente

Tabela 16 — Fluxos OIDC por tipo de cliente

Cliente	Fluxo OIDC	Observação
App mobile (Android)	<i>Authorization Code</i> + PKCE	O app não guarda segredo de cliente; o refresh token vai para o Android Keystore , nunca em <code>SharedPreferences</code> em texto claro
Web (JSP atual e novo front)	<i>Authorization Code</i> + cookie <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite=Lax</code>	Estado anti-CSRF obrigatório no <i>callback</i> ; sessão deixa de ser o mecanismo de identidade
Painel interno da adega / back-office	<i>Authorization Code</i> + segundo fator (TOTP) obrigatório para papéis <code>admin</code> , <code>gestor-adega</code> e <code>enologo</code>	Ação irreversível (baixa de estoque, laudo) exige 2FA
Serviço → serviço	<i>Client Credentials</i> (token de serviço de vida curta) + mTLS	Cada serviço tem sua própria identidade; não existe "rede interna confiável"
Rastreo público do lote (QR)	<i>endpoint</i> público, sem token , com resposta mínima (lote, safra, origem)	Informação pública por definição; nenhum dado pessoal

5.3.4 Comunicação entre serviços

- **mTLS** entre serviços e para o gateway (certificados curtos, rotação automática via *service mesh*): garante canal cifrado e identidade mútua — não basta cifrar e assumir que o emissor é quem diz ser.
- Autorização entre serviços por **escopo do token de serviço** (ex.: só o `ms-pedidos` tem o escopo `estoque:reservar`; só o `ms-qualidade` tem `notificacao:disparar`).
- **Broker MQTT/IoT com identidade por dispositivo** (corrige o modelo atual de credencial compartilhada): cada adega tem usuário próprio no broker (`adega-<id>`), com **ACL restrita a publicar** somente no seu tópico de telemetria — o dispositivo não pode assinar o próprio *namespace* de comandos nem publicar no tópico de outro dispositivo. A ponte Node-RED legada publica no tópico físico atual (`Fiap/iot/3ESOA/CodeSquad/sensor`); na migração (Strangler Fig), um *bridge* mapeia esse tópico para `qualidade.leitura` **sem exigir recompilar o firmware** — o dispositivo não muda, a política muda na borda.

5.3.5 Boas práticas de API aplicadas

Tabela 17 — Boas práticas de API aplicadas

Prática	Como é aplicada	Por quê
Rate limiting e quotas	No gateway, por <code>sub</code> + por serviço: 60 req/min por usuário, 600 req/min por cliente de serviço; <code>burst</code> com <code>token bucket</code>	Protege contra abuso, varredura de catálogo e negação de serviço com credencial válida
Validação de entrada contra o contrato	Todo payload validado contra o OpenAPI de cada serviço; rejeitar campo desconhecido em escrita	Impede <i>mass assignment</i> (o clássico "cliente manda <code>status</code> e <code>preco</code> no corpo do pedido")
Idempotência	<code>Idempotency-Key</code> obrigatório em POST de pedido, pagamento e movimentação de estoque; chave guardada 24 h com o resultado	Retry de rede e clique duplo não podem virar dois pedidos nem dois débitos
Limite de tamanho e paginação	Corpo ≤ 1 MB; <code>limit</code> obrigatório com teto de 100 itens	Evita consulta que derruba o serviço (<code>?limit=999999</code>)
Nada de dado sensível na URL	Identificador, nunca CPF/e-mail em <code>path</code> ou <code>query</code>	URLs aparecem em log de proxy, histórico e métrica
Erros sem vazamento	Resposta de erro padronizada (<code>codigo</code> , <code>mensagem</code> , <code>traceId</code>); sem <code>stack trace</code> nem SQL para o cliente	Erro detalhado é mapa para o atacante

OWASP API Security Top 10 (2023) — cobertura explícita:

Tabela 18 — OWASP API Security Top 10 (2023): risco e contramedida

Risco	Contramedida nesta arquitetura
API1 BOLA (acesso a objeto de outro usuário)	Autorização sempre no serviço dono do dado, com filtro por <code>sub</code> /proprietário no repositório; teste automatizado com dois usuários distintos
API2 Autenticação quebrada	OIDC + JWT curto + 2FA no back-office; senha nunca trafega para serviço de negócio
API3 Consumo irrestrito de recurso	Rate limit/quota no gateway, paginação obrigatória, timeout em toda chamada
API4 Consumo irrestrito de recurso (fluxo)	Limite de itens por pedido e por lote; fila com teto e <i>backpressure</i>
API5 Autorização em nível de função	Matriz de papéis (seção 5.4) aplicada por endpoint, não por tela
API6 Acesso irrestrito a fluxos sensíveis	Fluxo de pagamento só via <code>ms-pagamentos</code> , com <code>Idempotency-Key</code> e verificação antifraude
API7 SSRF	Serviços não buscam URL fornecida pelo cliente; <i>webhook</i> futuro só com lista de destinos permitidos
API8 Configuração incorreta	CORS com origem explícita (fim do <code>AllowAll</code>), <code>AllowedHosts</code> restrito, TLS obrigatório, Swagger desligado em produção
API9 Inventário desatualizado	Catálogo de serviços com dono + OpenAPI versionado no repositório (seção 5.9)
API10 Consumo de API de terceiros sem validação	Validação da resposta do adquirente de pagamento e falha segura em resposta inesperada

5.4 Autorização (RBAC + ABAC)

Modelo: RBAC como base (papel → permissão) com **regras ABAC** para os casos em que o papel sozinho não decide (loja, região, propriedade do dado).

Papéis reais da vinheria: `cliente`, `atendente`, `enologo`, `gestor-adega`, `admin`.

Tabela 19 — Matriz de autorização por papel e por serviço (RBAC + ABAC)

Serviço	cliente	atendente	enologo	gestor-adega	admin
---------	---------	-----------	---------	--------------	-------

ms-catalogo	ler	ler	ler	ler	ler, escrever
ms-clientes	ler/editar próprio perfil	ler/editar clientes da sua loja	—	—	ler, escrever
ms-pedidos	criar/ler próprios pedidos	ler/criar pedido para cliente, cancelar	—	—	total
ms-estoque	ler saldo público	ler saldo, reservar	—	ler, movimentar, ajustar	total
ms-lotes	ler rastreio público do lote	ler	ler, criar/validar lote	ler, movimentar	total
ms-producao	—	—	ler, registrar	registrar colheita	total
ms-qualidade	—	—	ler análises, emitir laudo	ler telemetria, configurar faixas/alertas	total
ms-fornecedores	—	consultar	—	consultar, receber	total
ms-pagamentos	pagar próprio pedido, estornar sob regra	registrar pagamento	—	—	estornar
ms-analytics	—	indicadores da sua loja	—	indicadores de adega	total

Exemplos de regra **ABAC** (o papel não basta):

- **enologo** **só altera** análise laboratorial enquanto o lote estiver no estado `em_analise` — depois, apenas leitura e a alteração exige justificativa registrada.
- **atendente** só vê clientes da **sua** loja (`cliente_loja == atendente.loja`) — é o mesmo problema de API1 (BOLA), resolvido por atributo e não por tela.
- **gestor-adega** só ajusta faixa de temperatura da **sua** adega (`adeгаId` presente no token/atributo).
- Alteração de preço de catálogo acima de um percentual exige **admin** (dupla aprovação: `admin + gestor-adeга`).

Como é aplicado (e onde não é): a autorização é decidida **no serviço dono do dado**, com o contexto vindo do token (papel + atributos). O gateway faz a **autorização grossa** (token válido? escopo da rota?) e o serviço faz a **autorização fina** (esse usuário pode esse objeto?). Nunca se decide autorização em tela ou em JavaScript de front-end.

Propagação: o gateway encaminha a identidade em cabeçalho assinado (`X-Identidade` + assinatura do gateway) junto do `traceId`; o serviço valida a assinatura do gateway, o que impede um cliente de forjar `X-Identidade` direto contra um serviço.

5.5 Escalabilidade

Tabela 20 — Escalabilidade: frente, decisão e justificativa

Frente	Decisão	Por que é necessário no nosso caso
Serviços <i>stateless</i>	Nenhum estado de sessão no processo; identidade em JWT, não em sessão de servidor	É a causa raiz de hoje: <code>InMemoryDatabase</code> guarda sessão e credencial no processo do Servlet — reiniciar derruba a sessão e duas instâncias divergem
Autoscaling horizontal	HPA por CPU (70%) e por RPS/requisição por pod; mínimo 2 réplicas nos serviços de núcleo	Pico sazonal: Dia dos Namorados e Natal concentram venda em poucos dias
Cache de leitura	Redis para catálogo (TTL 60 s) e para o rate limit distribuído	Catálogo é ~90% leitura; o <i>rate limit</i> precisa de contador compartilhado entre réplicas

Réplica de leitura	Uma réplica de leitura por serviço de núcleo; escrita no primário	Relatório e catálogo não competem com a escrita do pedido
Particionamento de eventos	Kafka particionado por chave: <code>loteId</code> (ordem preservada por lote) e <code>pedidoId</code>	Ordem por lote é regra de negócio (genealogia e rastreio); ordem global não é necessária
Telemetria da adega	Tópico particionado por <code>deviceId</code> , com escrita em banco de série temporal e <i>batching</i>	Com a leitura a cada 4 s (<code>INTERVALO_LEITURA_MS = 4000</code> no firmware), 50 adegas já geram ~12,5 mensagens/s contínuas, 24 h por dia
Banco por serviço	Um banco por serviço (PostgreSQL), MongoDB para catálogo, série temporal para telemetria	Substitui o SQLite de arquivo único compartilhado, que é o gargalo e o acoplamento atual
Migração gradual	<i>Strangler Fig</i> : o gateway passa a rotear domínio por domínio; o monolito continua respondendo o resto	Permite escalar por domínio sem reescrever tudo de uma vez

5.6 Resiliência

5.6.1 Padrões de falha aplicados

Tabela 21 — Resiliência: padrão de falha, aplicação concreta e parâmetro

Padrão	Aplicação concreta	Parâmetro
Timeout	Toda chamada síncrona tem prazo: catálogo 2 s, estoque 3 s, pagamento 5 s	Nenhuma chamada sem prazo; prazo do gateway sempre maior que o do serviço
Retry com backoff + jitter	Só em operação idempotente (<code>GET</code> e <code>POST</code> com <code>Idempotency-Key</code>)	3 tentativas, 200 ms → 400 ms → 800 ms + jitter aleatório
Circuit Breaker	Por destino: <code>ms-pagamentos</code> , <code>ms-estoque</code> ; adquirente externo	Abre com 50% de erro em ≥ 20 chamadas na janela de 30 s; <i>half-open</i> em 30 s
Bulkhead	<i>Pool</i> e limite de concorrência separados para <code>ms-pagamentos</code> e para a ingestão de telemetria	Pagamento lento não pode consumir as conexões do catálogo
Dead Letter Queue	Tópico <code><topico>.DLQ</code> com retenção de 7 dias e alerta	Mensagem envenenada sai da fila principal sem travar o consumo
Idempotência no consumidor	Toda mensagem tem <code>idEvento</code> ; consumidor guarda os últimos <code>idEvento</code> processados	At-least-once é o padrão real de mensageria; sem isso, evento repetido cobra duas vezes
Degradação elegante	Catálogo responde do cache Redis se o serviço cair; consulta de estoque devolve último saldo conhecido com aviso "saldo pode estar desatualizado"	Preferir resposta parcial explícita a erro total
Falha segura	Se o <code>ms-pagamentos</code> não responde, o pedido fica <code>aguardando_pagamento</code> e a reserva de estoque é liberada após o prazo	Nunca "assumir aprovado"

5.6.2 Fluxo de compensação (ligação com o Saga da seção 3)

Compra de vinho: gateway → `ms-pedidos` (`pedido.criado`) → `ms-estoque` reserva (`estoque.reservado`) → `ms-pagamentos` (`pagamento.aprovado` | `pagamento.recusado`) → confirma pedido → `ms-notificacoes` avisa → baixa definitiva.

Falha no pagamento: `pagamento.recusado` → `ms-estoque` **libera a reserva** → `ms-pedidos` marca `cancelado` → `ms-notificacoes` informa. A compensação é evento, não comando direto: o serviço que reservou é o único que pode liberar.

5.6.3 Continuidade e deploy

- **Blue-green/canário** por serviço, com *rollback* automático por SLO (aumento de erro 5xx ou de latência p95 na nova versão).
- **Migração de banco compatível** com a versão anterior (expandir → migrar → contrair), para que o *rollback* não exija restaurar banco.
- **Backup** diário por serviço + **teste de restauração** trimestral registrado (backup não testado não é backup).
- Metas de continuidade: **RPO 15 min / RTO 1 h** para *ms-pedidos*, *ms-estoque* e *ms-pagamentos*; RPO 1 h / RTO 4 h para os demais.

5.7 Observabilidade

- **OpenTelemetry** com *traceId* único gerado no gateway e propagado em HTTP e nos cabeçalhos das mensagens Kafka — é o que permite ligar "o cliente reclamou que o pedido sumiu" a uma cadeia de 5 serviços.
- **Métricas** (Prometheus) por serviço: latência p50/p95/p99, taxa de erro, saturação; por tópico: *lag* do consumidor e tamanho da DLQ; de negócio: pedidos criados, pagamentos aprovados, **alertas de adega**.
- **Dashboards e alertas** (Grafana): alerta de negócio (temperatura fora da faixa — *qualidade.alerta*), alerta de consumo (*lag* de tópico > 5 min), alerta de borda (taxa de 401/429 anômala, possível tentativa de varredura).
- **Logs estruturados** em JSON, com *traceId*, sem dado pessoal (seção 5.8).

SLOs propostos (mensais, por serviço):

Tabela 22 — SLOs propostos por serviço (mensais)

Serviço	Disponibilidade	Latência	Outro
API Gateway	99,9%	p95 < 250 ms	—
<i>ms-catalogo</i>	99,9%	p95 < 150 ms (cache)	—
<i>ms-estoque</i> (consulta)	99,9%	p99 < 400 ms	saldo correto é regra de negócio
<i>ms-pedidos</i>	99,9%	p95 < 500 ms	erro 5xx < 0,5%
<i>ms-pagamentos</i>	99,9%	p95 < 1,5 s	nenhuma cobrança duplicada (conferência diária)
<i>ms-qualidade</i>	99,5%	ingestão < 5 s da leitura	alerta crítico de adega < 30 s
<i>ms-analytics</i>	99,0%	atualização < 15 min	atraso não afeta venda

5.8 Segurança da informação, segredos e LGPD

5.8.1 Gestão de segredos

Tabela 23 — Gestão de segredos: hoje e na arquitetura proposta

Hoje	Como fica
Credenciais MQTT e cadeia de conexão em arquivo de configuração (<i>appsettings.json</i>) e no README (<USUARIO>/<SENHA>)	Segredos em cofre (Vault / <i>sealed secrets</i>), injetados em tempo de execução; nada de segredo em repositório, em imagem ou em log
Senha de usuário em texto claro na memória (<i>InMemoryDatabase.CREDENTIALS</i>)	Senha nunca é armazenada por serviço de negócio: quem autentica é o Keycloak (hash com Argon2id/bcrypt)
Token estático de API	Token de serviço de vida curta, emitido por <i>Client Credentials</i> , com rotação automática

Sem política de rotação	Rotação programada de segredos de broker, banco e adquirente (90 dias, no máximo) e imediata em caso de vazamento
-------------------------	---

5.8.2 LGPD

Tabela 24 — LGPD: item e tratamento

Item	Como tratamos
Base legal e finalidade	Cadastro e pedido = execução de contrato; campanha/marketing = consentimento explícito e granular , registrado com data e versão do texto
Minimização	A base analítica não precisa de nome, e-mail nem CPF: o <code>ms-analytics</code> recebe evento com identificador pseudonimizado (<code>clienteId</code> → <code>hash</code> por safra de chave), nunca o cadastro
O caso do CSV atual	<code>historico_vendas-vinheria_agnello.csv</code> traz <code>Idade_Cliente</code> : dado pessoal em base analítica. Na arquitetura proposta, idade entra agregada por faixa (18-24, 25-34, ...) e o bruto fica no <code>ms-clientes</code> , com acesso auditado
Direito do titular	Exclusão/anonimização no <code>ms-clientes</code> dispara evento <code>cliente.anonimizado</code> ; <code>ms-analytics</code> e <code>ms-notificacoes</code> reagem — sem depender de varredura manual
Retenção	Pedido fiscal: 5 anos. Telemetria de adega: 12 meses em série temporal, depois agregada. Log com dado pessoal: 30 dias
Logs e trace	CPF/e-mail/telefone mascarados (<code>***.***.***-**, a***@dominio</code>) na origem; <code>traceId</code> ≠ dado pessoal
Trilha de auditoria	Toda ação de <code>admin</code> , <code>gestor-adega</code> e <code>enologo</code> (alterar preço, ajustar estoque, emitir laudo, estornar) grava quem/quando/o quê/antes-depois, em <code>append-only</code>
Pagamentos (PCI-DSS)	Número de cartão nunca entra no nosso domínio: tokenização no adquirente; o <code>ms-pagamentos</code> guarda apenas <code>token</code> , bandeira, últimos 4 dígitos e status
Mobile	Banco Room cifrado (SQLCipher), token no Android Keystore, <code>cache</code> de estoque limpo no <code>logout</code>

5.8.3 Privacidade desde a concepção

Nenhum serviço recebe dado pessoal de que não precise: catálogo, estoque, qualidade e analytics trabalham por identificador. Quem conhece cadastro é o `ms-clientes` (e o Keycloak, para credencial). Isso reduz o alcance de qualquer incidente — vazamento em `ms-estoque` não expõe cadastro.

5.9 Governança

5.9.1 Contrato como fonte da verdade

- **OpenAPI 3.1** por serviço (`ms-*/contracts/openapi.yaml`) e **AsyncAPI** para os eventos (`contracts/asyncapi.yaml`), versionados **no repositório** — não gerados só em tempo de execução.
- **CI valida contrato**: `lint` (Spectral), quebra de compatibilidade detectada por `diff` e **build que falha** quando um campo obrigatório é removido, um tipo muda ou um `enum` perde valor. Contrato quebrado não chega ao `main`.
- Serviços mockados a partir do contrato desde o primeiro dia (permite Roger/Arthur escreverem contra contrato antes do serviço existir).

- Swagger desligado em produção (hoje só existe em `Development`, o que já é o comportamento correto — a diferença é o contrato deixar de ser só de desenvolvimento e passar a ser artefato versionado).

5.9.2 Versionamento e depreciação

- Versão no caminho (`/v1/...`), nunca no corpo ou em cabeçalho.
- Compatibilidade: adição de campo é compatível; remoção/renome/estreitamento de tipo **não é** — exige nova versão.
- Depreciação formal: anúncio, cabeçalho `Deprecation/Sunset`, **90 dias** de convivência, telemetria de uso da versão antiga antes de desligar. Nenhuma versão é desligada sem uso residual igual a zero.
- Eventos: evolução de esquema (*schema registry*) permite adicionar campo opcional; mudança incompatível cria novo tópico com sufixo de versão.

5.9.3 Catálogo de serviços e papéis de nome

Tabela 25 — Convenções de nome para serviços, tópicos, rotas, bancos e filas mortas

Convenção	Regra	Exemplo
Serviço	<code>ms-</code> + domínio, minúsculo, <i>kebab-case</i>	<code>ms-qualidade</code> , <code>ms-pagamentos</code>
Tópico de evento	<code>dominio.evento</code> , minúsculo, sem prefixo de turma	<code>pedido.criado</code> , <code>estoque.reservado</code> , <code>qualidade.alerta</code>
Chamada REST	<code>/v1/recurso</code> , substantivo no plural, sem verbo na URL	<code>GET /v1/vinhos</code> , <code>POST /v1/pedidos</code> (e não <code>/v1/listarVinhos</code> , <code>/v1/v1/...</code>)
Banco/esquema	<code>db_<serviço></code>	<code>db_estoque</code>
Fila morta	<code><topico>.DLQ</code>	<code>pedido.criado.DLQ</code>

Catálogo com **dono** por serviço (evita "serviço órfão" que ninguém mantém nem conhece):

Tabela 26 — Catálogo de serviços com dono funcional e responsável técnico no grupo

Serviço	Dono (papel funcional)	Responsável técnico no grupo
API Gateway, <code>ms-identidade</code>	Arquitetura	Roger (seção 1) com revisão da Yasmin (seção 5)
<code>ms-catalogo</code> , <code>ms-producao</code> , <code>ms-lotes</code> , <code>ms-estoque</code> , <code>ms-fornecedores</code>	Operação da vinheria	A definir na seção 1 (lista congelada em 21/09)
<code>ms-pedidos</code> , <code>ms-pagamentos</code> , <code>ms-notificacoes</code>	Comercial	Arthur (seção 4)
<code>ms-qualidade</code>	Adega	Kevin (seção 2)
<code>ms-clientes</code> , <code>ms-analytics</code>	Dados e LGPD	Yasmin (seção 5)

Os responsáveis técnicos seguem a divisão do cronograma da Fase 6; os domínios definitivos vêm da lista congelada da seção 1, que é a fonte única dos nomes.

5.9.4 ADRs — Architecture Decision Records

Arquivos curtos em `Documentacao/Fase6/adr/`, um por decisão: contexto → decisão → consequências (boas e ruins).

- **ADR-001 — OAuth 2.0/OIDC com Keycloak no lugar de sessão de servidor.** Contexto: o monolito autentica com sessão e credenciais em memória (`InMemoryDatabase`), o que não escala nem sobrevive a *restart*. Decisão: identidade centralizada, JWT de vida curta, nenhum

serviço guarda senha. Consequência: serviços ficam *stateless* e escaláveis; em troca, o *ms-identidade* vira dependência crítica (mitigada com cache de JWKS e tolerância a falha na validação).

- **ADR-002 — Validação de token centralizada no API Gateway.** Contexto: validar em todos os serviços duplicaria lógica e acoplaria cada serviço ao Keycloak. Decisão: validação de borda + canal interno autenticado por mTLS. Consequência: menos duplicação e um ponto de política; em troca, o gateway vira ponto único (mitigado com réplicas e *health check*).
- **ADR-003 — Identidade MQTT por dispositivo com ACL de publicação.** Contexto: credencial compartilhada com Publish+Subscribe permite leitura falsa de adegas. Decisão: usuário por dispositivo, publicação restrita ao tópico próprio, ponte de compatibilidade para o tópico legado. Consequência: telemetria confiável e auditável; em troca, há que provisionar credencial por dispositivo e manter a ponte durante a migração.

5.10 Riscos residuais e pendências

Tabela 27 — Riscos residuais e pendências

Item	Situação	Depende de
Nomes definitivos dos serviços e donos do dado	Estimados a partir do repositório e da proposta da seção 1	Lista congelada da seção 1 (21/09) — esta seção usa os mesmos nomes assim que congelados
Papéis da matriz de autorização por serviço	Definidos por domínio funcional	Revisão final contra o diagrama da seção 2 e a matriz da seção 4
Nomes dos tópicos Kafka	Propostos aqui e iguais aos do desenho	Confirmação cruzada com a seção 4 (fonte única: <i>pedido.criado</i> , <i>estoque.reservado</i> , <i>pagamento.aprovado</i> , <i>pagamento.recusado</i> , <i>lote.criado</i> , <i>qualidade.leitura</i> , <i>qualidade.alerta</i> , <i>notificacao.enviar</i> ; eventos de privacidade: <i>cliente.anonimizado</i>)
Cifra do banco Room e fluxo PKCE no mobile	Especificado nesta seção	Execução depende do time mobile (fase de implementação, não desta atividade)
Política de retenção formal (parecer jurídico)	Valores propostos com base em prática de mercado e LGPD	Validação do professor/instituição, se exigida

5.11 Fechamento

Esta seção fecha a arquitetura pelo lado que o desenho sozinho não garante: **quem pode fazer o quê, sob quais condições, com qual rastreabilidade e o que acontece quando algo falha**. Ela responde, com decisões explícitas e justificadas, aos pontos que a auditoria do próprio repositório expôs: sessão e senha padrão em memória (seção 5.3), rotas administrativas sem verificação de papel (seção 5.4), CORS aberto e API sem autenticação efetiva (seção 5.3.5), banco único compartilhado (seção 5.5), contrato não versionado (seção 5.9) e credencial de broker compartilhada (seção 5.6, ADR-003).

Para a consolidação do *.docx*, esta parte entra como **Seção 5** na ordem exigida pelo enunciado, com a tabela de divisão de tarefas já prevista. Nomes de serviços, de tópicos e de papéis aqui usados devem permanecer idênticos aos das Seções 1 a 4 — qualquer mudança posterior exige atualização nas cinco seções.

Apêndice A — Divisão de tarefas

Tabela 28 — Divisão de tarefas por item da atividade

Item da atividade	Seção	Responsável	RM	Entregável no repositório	Período
1 — Identificação dos microsserviços	1	Roger Viana Gonçalves de Alencar	97540	<code>Documentacao/Fase6/P1_servicos.md</code>	16/09 a 21/09/2026
2 — Diagrama de arquitetura	2	Kevin Benevides da Silva Romariz	557898	<code>P2_arquitetura.d</code>, <code>.md</code>, <code>.png</code> e <code>.svg</code>	22/09 a 28/09/2026
3 — Padrões e justificativas	3	André Luiz dos Santos Flores	554952	<code>P3_padroes.md</code>	22/09 a 30/09/2026
4 — Integração entre serviços	4	Arthur Cei Corrêa	97781	<code>P4_integracao.m</code>	29/09 a 05/10/2026
5 — Segurança e governança (Parte 1)	5	Yasmin Kimura	557413	<code>P5_seguranca_g</code> + <code>adr</code>	06/10 a 24/09/2026
5 — Consolidação do documento (Parte 2)	capa, sumário e apêndices	Yasmin Kimura	557413	este documento	06/10 a 13/10/2026 (previsto)

A revisão cruzada — cada integrante revisando a seção de outro — é a etapa seguinte do cronograma, prevista para 11/10 a 13/10/2026, antes do fechamento da versão final e do envio.

Apêndice B — Registro de revisão consolidado

Os registros abaixo são reproduzidos do material de origem de cada seção. Neles, as referências a P1 a P5 correspondem às seções 1 a 5 deste documento, e as remissões internas citadas seguem a numeração do arquivo de origem, não a deste documento.

Seção 1 — Identificação dos Microserviços e Fronteiras de Domínio

- **v1.0 (21/09/2026)** — conteúdo original do item 1, entregue em PDF (Roger Viana Gonçalves de Alencar).
- **v1.1 (22/09/2026)** — revisão de consolidação (Yasmin Kimura, Pessoa 5): reconciliação com a lista de partida, definição de posse de *Safra* e de *MovimentacaoEstoque*, quem escreve o movimento de entrada do recebimento, fronteiras justificadas para *ms-clientes*, *ms-fornecedores* e *ms-analytics*, caminhos completos dos arquivos do repositório e conversão para *P1_servicos.md*.

Seção 2 — Diagrama de Arquitetura

- **v1.0 (29/09/2026)** — diagrama, legenda e texto do item 2 (Kevin Benevides, branch *docs/fase6-p2-arquitetura*, PR #3).
- **v1.1 (29/09/2026)** — revisão de consolidação (Yasmin Kimura, P5): páginas de detalhe 1–3 no *.drawio* e seus *.png/.svg* (critério "fonte ≥ 12 no tamanho final"), medições de impressão no item 2.4, citações de caminho do repositório no item 2.1 (regra 4 do *README.md* da fase) e correção do rótulo do monolito (a API .NET tem banco SQLite **próprio**, configurado em *appsettings.json*; o web JSP usa *InMemoryDatabase.java* — não há banco compartilhado entre os dois).
- **v1.2 (01/10/2026)** — revisão de consolidação (Yasmin Kimura, P5): corrigida a contradição do item 2.5 sobre o *ms-notificacoes* (ele assina *cliente.anonimizado*, que apenas limpa preferência de canal, e só dispara envio por *notificacao.enviar*); registrada a conferência cruzada com o P4 (01/10/2026), com a decisão sobre *lote.criado* e a pendência de desenho das chamadas síncronas internas que a matriz do P4 lista.
- **v1.3 (01/10/2026)** — revisão de consolidação (Yasmin Kimura, P5): desenhadas na página 1 as três chamadas síncronas internas que faltavam (*ms-pedidos* → *ms-catalogo*, *ms-pedidos* → *ms-clientes* e *ms-pagamentos* → *ms-pedidos*), em setas azuis contínuas com rótulo, completando as quatro relações síncronas da matriz do P4; *.png* e *.svg* reexportados com os mesmos parâmetros do item 2.4 (tamanho e menor fonte medidos novamente: 3992 × 3126 px e 8,6 pt, sem alteração); acrescentado o caminho do CLI do draw.io no Windows.

Seção 3 — Padrões de Arquitetura e Justificativas

- **v1.0 (30/09/2026)** — versão inicial do Item 3, com os quatro padrões obrigatórios e três padrões adicionais consistentes com o P1 e o P2 (André Luiz dos Santos Flores).
- **v1.1 (30/09/2026)** — revisão de consolidação (Yasmin Kimura, P5): Service Discovery justificado pelo ambiente-alvo (plataforma orquestrada, *Service* + DNS interno) em vez de manter a tecnologia indefinida; fronteira explícita com a seção 6 do P5 na lista de padrões não incluídos, para o documento não afirmar que Circuit Breaker, Bulkhead e idempotência estão fora da arquitetura.

Seção 4 — Integração entre Serviços: Comunicação Síncrona x Assíncrona

- **v1.2 (01/10/2026)** — revisão editorial para alinhar cabeçalho ao padrão das demais partes e distinguir a proposta de envelope de evento de um contrato já implementado.

- **v1.3 (01/10/2026)** — revisão de consolidação (Yasmin Kimura, P5): conferência cruzada da matriz e do catálogo contra a página 3 do `P2_arquitetura.drawio`; `lote.criado` alinhado ao P1 (item 1.4 — Lote, Genealogia e QR são do `ms-lotes`) e ao P2; `cliente.anonimizado` com o consumidor `ms-pedidos` restaurado; incluídas as linhas `API Gateway → monolito atual` (Strangler Fig) e `ms-pedidos → ms-estoque`; linha de autenticação corrigida para o fluxo OIDC do P5 (a emissão de token não passa pelo gateway); caminhos de arquivo no fluxo IoT (regra 4 do `README.md` da fase); nova seção 5 com a conferência dos critérios de aceite.
- **v1.4 (01/10/2026)** — as três chamadas síncronas internas que faltavam no desenho foram acrescentadas à página 1 do `P2_arquitetura.drawio` (itens 2.1 e 2.5 do P2), então a matriz e o diagrama passam a mostrar as mesmas quatro relações síncronas entre serviços.

Apêndice C — Como regenerar as figuras

Exportação das páginas do diagrama

Abrir `P2_arquitetura.drawio` no draw.io (desktop ou `app.diagrams.net`). A página 1 é a visão geral; as páginas 2 a 4 são os detalhes. No Windows o CLI é o binário "`C:\Program Files\draw.io\draw.io.exe`" (é o mesmo `drawio` dos exemplos abaixo). Para exportar uma página:

```
# PNG (a opção -p é 1-based; NÃO usar --theme: na versão 30.3.6 ela faz o CLI
# responder "input file/directory not found" e não gerar arquivo)
drawio -x -f png -b 10 -s 2 -p 1 -o P2_arquitetura.png P2_arquitetura.drawio
drawio -x -f png -b 10 -s 2 -p 2 -o P2_arquitetura_d1_servicos.png P2_arquitetura.drawio
drawio -x -f png -b 10 -s 2 -p 3 -o P2_arquitetura_d2_eventos.png P2_arquitetura.drawio
drawio -x -f png -b 10 -s 2 -p 4 -o P2_arquitetura_d3_iot_migracao.png P2_arquitetura.drawio
```

Para o `.docx`, **não** usar o "Exportar como SVG" padrão do draw.io: ele grava o texto em `foreignObject` (HTML) — no arquivo da visão geral são 210 ocorrências —, que o Word não renderiza ("Text is not SVG - cannot display"). O SVG precisa ter o texto em curvas. O caminho com `pdftocairo` (usado na primeira versão) exige poppler instalado; onde ele não está disponível, o MuPDF/PyMuPDF faz o mesmo a partir do PDF de todas as páginas:

```
drawio -x -f pdf --all-pages --crop -o P2_arquitetura.pdf P2_arquitetura.drawio
python -c "import fitz; d=fitz.open('P2_arquitetura.pdf'); \
[open(n, 'w', encoding='utf-8').write(d[i].get_svg_image(text_as_path=True)) \
 for i,n in enumerate(['P2_arquitetura.svg', 'P2_arquitetura_d1_servicos.svg', \
 'P2_arquitetura_d2_eventos.svg', 'P2_arquitetura_d3_iot_migracao.svg'])]"
rm P2_arquitetura.pdf
```

Conferência depois de reexportar: o `.svg` não pode ter `foreignObject` e precisa ter `viewBox`; abrir o arquivo no navegador e conferir que nenhum texto saiu da caixa.